# Task Offloading for CAVs Edge Computing Environment: Taxonomy, Critical Review, and Future Road Map

BHOOPENDRA KUMAR, School of Computer Science Engineering and Technology (SCSET), Bennett University, Greater Noida, India
ADITYA BHARDWAJ, School of Computer Science Engineering and Technology (SCSET), Bennett University, Greater Noida, India
DINESH PRASAD SAHU, School of Computer Science Engineering and Technology (SCSET), Bennett University, Greater Noida, India

The rapid advancement of Intelligent Transportation Systems (ITS) has led to a paradigm shift toward the adoption of Connected Autonomous Vehicles (CAVs). In recent years, CAVs have emerged as a prominent research focus due to their potential to reduce road traffic accidents caused by human error, optimize traffic flow, create new economic opportunities, and enhance travel convenience. However, the increasing demand for compute and delay-sensitive applications, such as real-time navigation and sensor data processing, exceeds the capabilities of current onboard vehicle resources. Consequently, task offloading has gained significant attention, allowing certain computational tasks generated by CAVs operations to be offloaded to external cloud or edge servers. The existing review literature has been limited in its focus on task offloading techniques specifically for CAVs architecture. Therefore, this study aims at presenting a comprehensive survey on task offloading in CAVs through a systematic review guided by key research questions. We first provide a technical background and then propose a broad coverage taxonomy of existing literature, analyzing promising solutions such as Machine Learning (ML) and heuristic-based techniques. In addition, we present a taxonomy of execution environments, metrics, and datasets. Finally, we highlight key research challenges and future trends, providing valuable insights for advancing task offloading in CAVs architecture.

CCS Concepts: • **Computing methodologies → Machine learning;**

Additional Key Words and Phrases: Connected Autonomous Vehicles (CAVs), edge computing, task offloading, federated learning, deep reinforcement learning (DRL), offloading execution environment

**ACM Reference Format:**
Bhoopendra Kumar, Aditya Bhardwaj, and Dinesh Prasad Sahu. 2026. Task Offloading for CAVs Edge Computing Environment: Taxonomy, Critical Review, and Future Road Map. *ACM Comput. Surv.* 58, 8, Article 194 (February 2026), 35 pages. https://doi.org/10.1145/3783984

Authors' Contact Information: Bhoopendra Kumar, School of Computer Science Engineering and Technology (SCSET), Bennett University, Greater Noida, Uttar Pradesh, India; e-mail: e22soep0051@bennett.edu.in; Aditya Bhardwaj (corresponding author), School of Computer Science Engineering and Technology (SCSET), Bennett University, Greater Noida, Uttar Pradesh, India; e-mail: aditya.cse@nitttrchd.ac.in; Dinesh Prasad Sahu, School of Computer Science Engineering and Technology (SCSET), Bennett University, Greater Noida, Uttar Pradesh, India; e-mail: dinesh.sahu@bennett.edu.in.
Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM 0360-0300/2026/02-ART194
https://doi.org/10.1145/3783984

## 1 Introduction

This section discusses CAV market trends, high-level system architecture, and the growing need for task offloading. Furthermore, we highlighted key motivations, challenges, and formulated research questions to guide the systematic literature review.

### 1.1 Introduction to CAVs and Market Trends

People have been imagining autonomous or self-driving cars for the past hundred years. Francis P. Houdina drove a radio-controlled car through the streets of New York City in 1925, making initial attempts to develop a driverless car [36]. For autonomous vehicles to operate, their intelligence mostly depends on the information they gather about pedestrians and nearby vehicles, [5, 100]. Three important requirements driving the necessity for autonomous vehicles are: (i) reduce traffic congestion, (ii) increase user experience, and (iii) enhance road safety by reducing human mistakes and accidents. The U.S. Department of Transportation's **National Highway Traffic Safety Administration** (**NHTSA**) has issued a report for 2024, indicating that around 39,345 people died in traffic accidents. NHTSA also estimates that the majority of these traffic-related fatalities were caused by human error. These alarming trends underscore the urgency of deploying advanced vehicle technologies that can mitigate human error, responsible for the majority of traffic accidents [66]. For example, Waymo states that its autonomous driving systems are built to operate independently of human impairments, thereby greatly enhancing their safety.

Furthermore, a global research report projected that the CAVs market will reach USD 614.88 billion by 2030, growing at a **Compound Annual Growth Rate** (**CAGR**) of 24.9% from 2023 to 2030, thus transforming the advancement of **Intelligent Transportation Systems** (**ITS**) [85] **[RQ1.3]**. This growing interest and market competition are driving innovation and accelerating adoption [76, 102]. The growing market demand for **Connected Autonomous Vehicles** (**CAVs**) has driven advances in vehicle functionality, requiring real-time decision-making and data processing via on-board sensors. However, limited computing resources onboard create bottlenecks for computation-intensive tasks. Task offloading to edge servers or **Roadside Units** (**RSUs**) becomes crucial to handle these tasks, ensuring low latency, conserving vehicle resources, and supporting the scalability of CAVs in ITS. As CAV adoption increases, efficient task offloading is essential to meet growing computational demands [109].

Therefore, to further deepen the understanding of the task offloading ecosystem in CAVs, we have drawn a high-level architecture framework as illustrated in Figure 1. This shows how the sensing, communication, decision-making and application layers work together to allow CAVs to drive themselves and connect to their surroundings. The CAV's architecture enables task offloading through multiple layers. The *sensing layer* collects data for offloading, while the *communication layer* ensures data exchange between vehicles and infrastructure. When needed, the *decision-making layer* offloads complex AI/ML tasks to cloud/MEC. The *onboard processing layer* handles immediate tasks, but offloads intensive computations externally. The cloud/MEC layer provides high computational power for complex analyses, ensuring efficient performance and real-time decision-making. This distributed system optimizes task offloading in CAVs.

### 1.2 Task offloading in CAVs: Motivation and Challenges

*1.2.1 Motivation.* Technology improvements have led to a new era of transportation systems, called CAVs, that require high processing, low latency, and improved user experience. Various computational activities, including autonomous and assisted driving, image-aided navigation, and augmented reality, are generated by vehicular driving systems and mobile devices to enhance vehicular services [10]. In CAVs environments, applications on varied devices generate massive volumes of data that user devices cannot analyze due to limited processing resources. CAVs primarily rely on

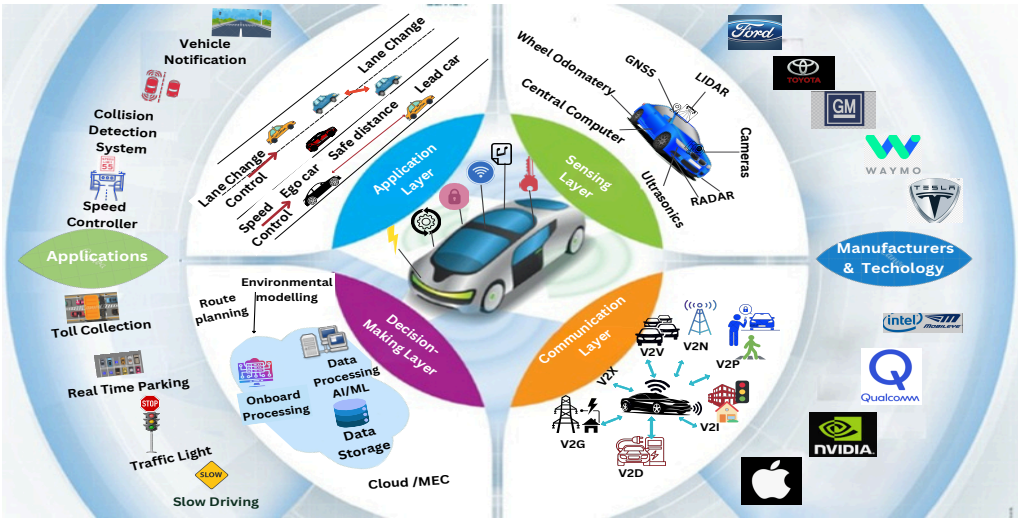



Fig. 1. Quick glimpse high-level architecture of CAVs.

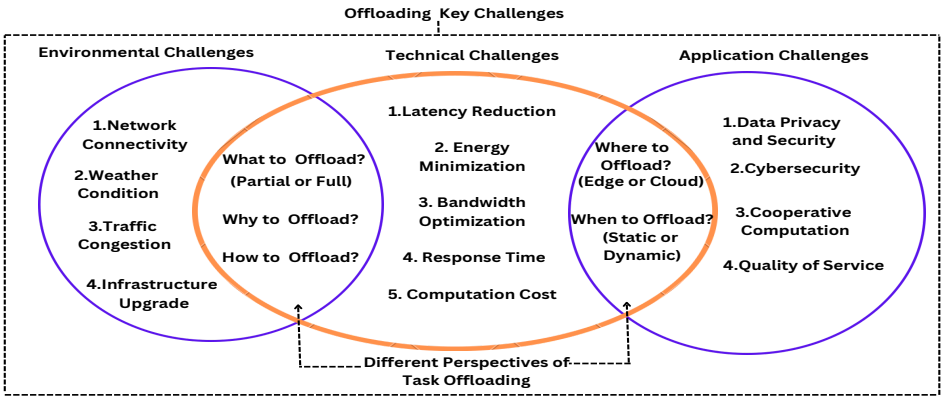



Fig. 2. Different perspectives and considerations for the design of the offloading mechanism.

real-time decision-making and data processing, requiring them to use low-latency computing. The answer to this problem involves offloading computing and improving communication infrastructure [23]. These key challenges and factors drive our motivation to review the evolving landscape of CAVs from a perspective of task-offloading techniques.

*1.2.2 Offloading Challenges.* The task offloading mechanism faces several challenges that must be addressed for efficient implementation. These challenges are categorized into three distinct groups: technical challenges, such as latency and bandwidth limitations; environmental challenges, such as network coverage and availability of the infrastructure; and application challenges, including concerns about data privacy and security **[RQ2.2]**. Each category covers essential aspects of the offloading decision process, which involves answering key questions of *what*, *where*, *when*, *why*, and *how* to offload tasks, as illustrated in Figure 2. Motivated by these factors and challenges, we have formulated research questions that aim at emphasizing a systematic review of the literature.

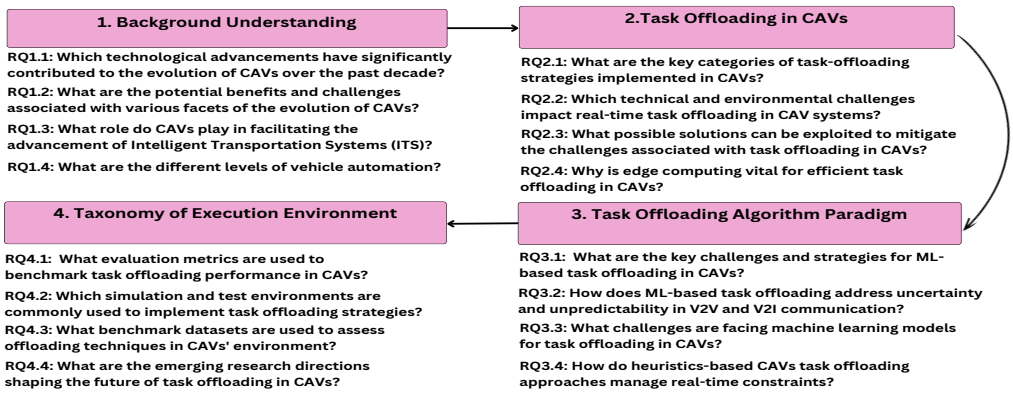



Fig. 3. Category-wise research questions formulation.

### 1.3 Formulated Research Questions For the Systematic Literature Review

The formulated research questions have been classified into four distinct categories RQ1-RQ4, as illustrated in Figure 3.

**RQ1** aims at exploring the key technological advancements, such as machine learning, V2X communication, sensor integration, and edge computing, that have emerged in CAVs over the past decade, and evaluate their influence on autonomous operation, decision-making, and system performance. **RQ2** explores the necessity of task offloading for CAVs, the challenges, and possible solutions. **RQ3** investigates how machine learning and heuristic-based approaches are employed to optimize task offloading strategies. Finally, **RQ4** identifies and analyzes the key performance metrics, benchmarking datasets, simulation tools, and evaluation frameworks used to assess the effectiveness of task offloading solutions in CAVs.

### 1.4 Existing Surveys and Our Contributions

Existing literature has focused on a single type of algorithmic solution paradigm, such as **Federated Learning** (**FL**), **Reinforcement Learning** (**RL**), and heuristic methods. Our work conducted a thorough critical analysis based on research questions addressing gaps and limitations in previous studies. Our comprehensive study offers an extensive review of task offloading in CAVs, presenting a proposed taxonomy that integrates machine learning and heuristic approaches as key solution paradigms, along with a detailed examination of execution environments, benchmarks, and datasets, which have not been thoroughly addressed in previous research. Table 1 presents a comparison of key contributions between the existing literature and our survey.

**The main contributions of this survey are outlined as follows**:

- We provide a high-level architectural framework for CAVs that serves as a foundation and offers valuable insights for connected autonomous vehicle systems.
- We perform a comprehensive analysis of the factors and challenges in designing the offloading mechanism and formulate research questions to explore the current literature landscape.
- We present a comprehensive analysis and taxonomy of proposed machine learning and heuristic-based task offloading solutions in CAVs edge computing systems.
- We provide a taxonomy of state-of-the-art offloading metrics, simulation tools, and datasets used, offering a reference for implementation environments.
- We identify gaps in the literature related to task offloading and highlight future research challenges in CAVs systems.

Table 1. Comparative Analysis of Existing Survey and our Contributions

| Article | | [16] | [65] | [77] | [95] | [82] | [15] | [92] | [125] | [88] | [21] | [128] | Our Work |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Year | | 2020 | 2020 | 2021 | 2021 | 2022 | 2022 | 2023 | 2023 | 2023 | 2024 | 2024 | 2025 |
| Taxonomy Coverage | Task Offloading | ✓ | ✓ | | ✓ | ✓ | | ✓ | ✓ | | ✓ | ✓ | ✓ |
| | Execution Environment | | | | | | | | | ✓ | | | ✓ |
| | Evaluation Metrics | | | | | | | | | | ✓ | | ✓ |
| Application | | ✓ | | ✓ | | ✓ | ✓ | | ✓ | ✓ | ✓ | ✓ | ✓ |
| Evolution of CAVs | | | | ✓ | | | ✓ | | | ✓ | | | ✓ |
| Level of Automation | | | | | ✓ | | ✓ | | | | ✓ | | ✓ |
| Architecture | | ✓ | ✓ | ✓ | | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | | ✓ |
| Algorithm Solution Paradigm | FL | | ✓ | | | | | | | | ✓ | | ✓ |
| | RL | ✓ | | | | ✓ | | | ✓ | ✓ | | ✓ | ✓ |
| | DRL | ✓ | ✓ | | | ✓ | | | ✓ | ✓ | | ✓ | ✓ |
| | Heuristic | | | ✓ | ✓ | | | | | | | ✓ | ✓ |
| Critical Analysis by RQ | | | | ✓ | | ✓ | | | | | | | ✓ |

### 1.5 Survey Structure

Figure 4 outlines the survey structure: Section 2 covers CAVs' evolution and automation levels, Section 3 presents a proposed edge computing-driven CAV framework. Section 4 introduces a taxonomy for task offloading, including objectives, types, servers, and techniques. Sections 5 and 6 detail the ML and heuristic-based offloading methods, respectively. Section 7 discusses execution environments, metrics, and datasets. Section 8 highlights the research challenges and future directions, and Section 9 concludes this survey.

### 1.6 Research Methodology

In the process of our article search strategy, we searched articles that contain the terms "CAVs task offloading in edge computing AND ML-based technique for CAVs." We submitted this search criteria to the digital scholarly databases. The article search and selection methodology used in this study is divided into four distinct stages. In the first phase we identify the articles using suitable keywords, then in 2nd phase, we do an Inclusion and Exclusion process, then in 3rd phase, we filter the articles based on our objective, and finally, in the last phase, we add the quality full article for our comprehensive review, which is shown in Figure 5 using PRISMA method.

## 2 CAVs Background: Technical Aspects, Issues, and Emerging Trends

This section presents the technical background of CAVs, detailing their evolution, levels of automation, and recent technological advancements to offer deeper insights into the current trends shaping the field.

### 2.1 Evolution of Connected Autonomous Vehicle (CAVs)

CAVs have evolved from basic driver assistance systems to completely autonomous vehicles with advanced **Vehicle-to-Everything** (**V2X**) communication, AI, and sensor technology for safer, more efficient, and connected transportation [74]. In the following, we have discussed the evolution of CAVs over a period of time **[RQ1.1]**.

The initial autonomous car was made by Houdina Radio Control in **1925**, which employed an antenna for receiving, but it needed an individual driver to monitor constantly [96]. In **1966**, the autonomous car Citroen DS was designed by a transport research laboratory from the UK, in which cables were fixed on the roadside for communication, but costs were too high [97]. Furthermore, in **1998**, Yann Lecun introduced **Convolutional Neural Networks** (**CNNs**), with the capability of an ANN model to automatically detect human handwriting patterns in an image using training examples instead of manual [2]. The year **2009** marked the independence of Waymo,

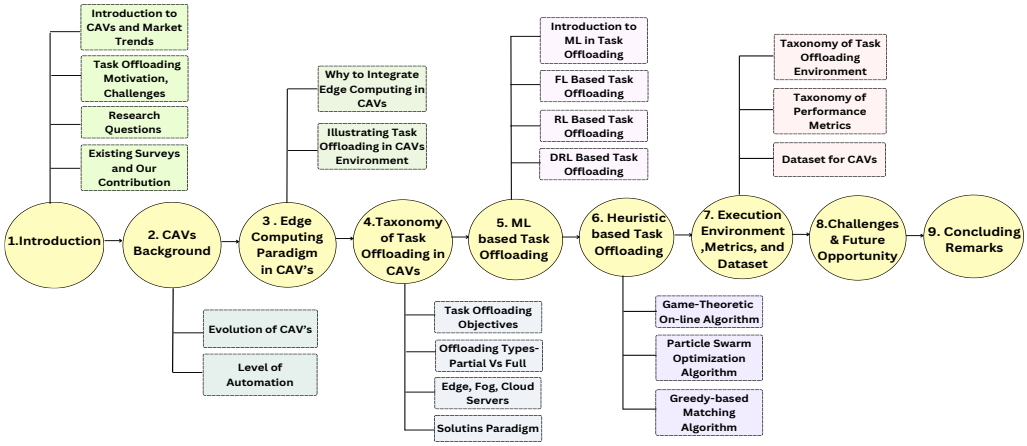



Fig. 4. Structure of the survey.

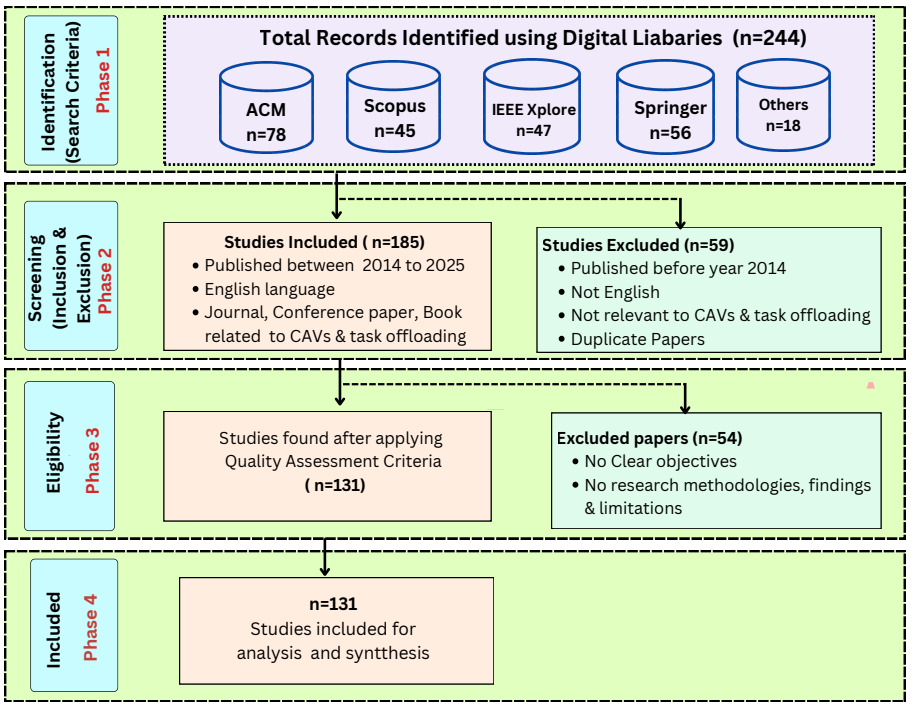



Fig. 5. PRISMA flow diagram for literature search criteria.

Google's driverless car project, under Alphabet Inc. Waymo continues to be a leading innovator in autonomous vehicle technology [116].

In **2016**, NVIDIA unveiled a fully driverless vehicle. The vehicle featured three cameras: left, right, and in the centre, positioned for input recording. Since **2020**, Honda has been working on fully autonomous cars in Japan with high-resolution maps, lane recognition, smart driver-support systems [45]. Ford began mass-producing an SAE Level-4 car in **2021**, followed by Cruise launching




Fig. 6. Evolution of connected autonomous vehicles.

a self-driving ride-hailing service in San Francisco in **2022**. In **2023**, Waymo demonstrated the practical feasibility of self-driving cars, and in **2024**, Mercedes-Benz introduced its advanced drive pilot technology [57].

In Figure 6, we have highlighted the evolution of CAVs over time. As CAVs progress, the complexity and data processing needs grow, driving the demand for task offloading. For example, earlier vehicles with basic radar/camera sensors can rely on on-board processing, while advanced models (e.g., Waymo's Chrysler Pacifica, Mercedes E-Class) would offload intensive tasks like traffic management, HD mapping, and autonomous driving control to the cloud or edge computing **[RQ1.2]**. Thus, as vehicle capabilities evolve, task offloading becomes crucial for handling increased computational demands in real-time.

### 2.2 Levels of Automation

Vehicle automation is categorized by the United States Department of Transportation's NHTSA [86] into five levels, ranging from Level 0 to 5, as shown in Figure 7 **[RQ1.4]**.

- **Level 0 (Not automated driving):** In Level 0, vehicles are operated manually, with the driver fully responsible for executing dynamic driving tasks.
- **Level 1 (Driver support):** In Level 1, a vehicle only has one automated system (cruise control) for acceleration and steering like a 2013 Ford Fusion with adaptive cruise control. For example, the Toyota Corolla was among the first vehicles to feature Level 1 autonomy, introduced in Japan in 2014 [54].
- **Level 2 (Partially driven automation):** Level 2 includes sophisticated technologies for driver assistance. The car has steering, acceleration, and deceleration controls. Tesla Model 3 and Mercedes-Benz E-Class introduced SAE Level 2 autonomy in Germany during 2016–2017, enabling partial driving automation under driver supervision [80].
- **Level 3 (Automated driving under conditions):** Currently, a vehicle can monitor its environment and take independent actions, such as racing past immovable objects. But even so, they still require a human override. In the event that the machine is unable to do the work, the driver must stay vigilant and prepared to take over, like the 2018 Audi A8 with its Traffic Jam Pilot feature [103].
- **Level 4 (High-level driving):** Vehicles equipped with this level of automation can function autonomously. However, unless rules and the environment change, they can only accomplish

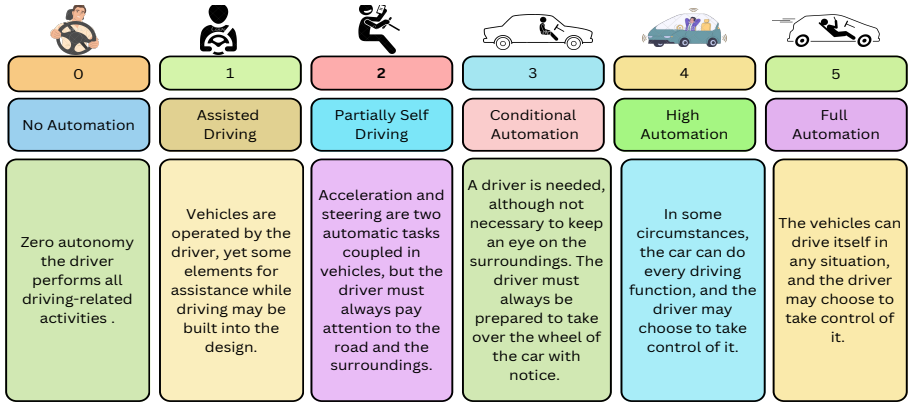



Fig. 7. Levels of vehicle automation.

this in a small region. For instance, Chrysler Pacifica Hybrid minivans equipped with Waymo's self-driving technology were first deployed for public ride services in Arizona in 2018 [131].

— **Level 5 (Fully autonomous):** Vehicles can function in any location or situation without the need for human involvement. With their sophisticated AI, sensor systems, and connections, they can independently traverse challenging situations. In 2023, Zoox and Amazon conducted the first robotaxi operation in Foster city, using autonomous vehicles as shuttle buses for city workers. Meanwhile, Tesla is advancing the development of a fully driverless taxi, designed without a steering wheel, brake, or accelerator pedal [6].

## 3 Edge Computing Paradigm in CAVs

This section introduces the edge computing paradigm in the context of CAVs, highlighting its role in addressing real-time processing demands. It explains the need for edge integration and presents a proposed architecture that illustrates the task-offloading process within edge-driven CAV environments.

### 3.1 Why to Integrate Edge Computing in CAVs?

The CAV design architecture is extremely complex, as it comprises key technologies including sensors, LiDAR, radar, GPS, navigation map, perception module, ultrasound, and so on. [42]. To process real-time data for an autonomous vehicle, guaranteeing the latency constraint is of paramount concern. For example, in autonomous intersection management, urban navigation and traffic management need fast processing and quick decision-making [124]. Therefore, to process complex computation of real-time heterogeneous data, delivering sufficient computing power is a very important requirement. Similarly, with the help of edge computing, the impact of computational offloading can be improved for IoT applications. This is because the battery capacity and other resources of IoT devices are typically limited. Task offloading to IoT devices mostly concentrates on minimizing job execution time, energy usage, and response time. Applications connected to industry, energy, smart cities, the health sector, and farming are just a few of the IoT use cases that stand to gain from task offloading. The IoT has always relied heavily on cloud-centric strategies for task offloading and processing the data. It's also used for the analytics of large-scale data coming from many IoT devices. However, the significant latency associated with cloud processing, along with the emergence of new mission-critical IoT applications, has driven both industry and academia

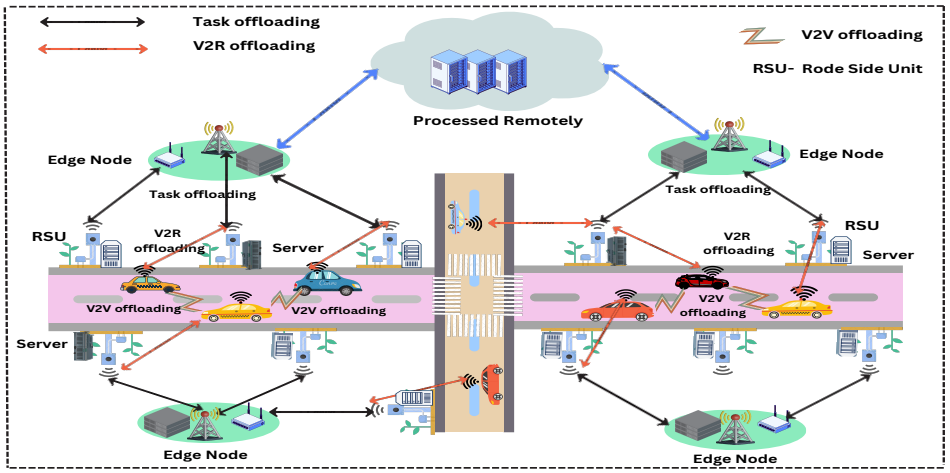



Fig. 8. Proposed architecture to illustrate the task offloading process in edge-driven CAVs environment.

to adopt edge computing solutions. As a result, local IoT clouds have developed with the intention of extending the battery life of the devices and increasing the quantity of offloaded activities that may be completed in close proximity to the IoT devices. However, the scalability problem of the IoT market, which mostly consists of dozens of billions of devices, often necessitates a cloud-edge collaboration while task offloading [30].

Edge computing has been a game changer in cloud and fog computing with respect to the CAV environment **[RQ2.4]**. Edge computing in CAVs task offloading leverages principles such as low latency, data proximity, resource efficiency, and mobility support to enable secure, real-time processing near vehicles. As shown in Figure 8, its architecture incorporates edge nodes, RSUs, and V2X communication modules for localized data handling, minimizing dependence on remote cloud servers. This significantly improves responsiveness, supports seamless task execution during vehicle movement, and enhances decision-making accuracy. By processing data near the source, edge computing also minimizes the network congestion and latency [33]. The performance of the automated vehicles can be improved by sharing the information from their closing vehicles with the assistance of edge computing concepts.

### 3.2 Proposed Architecture to Illustrate Task Offloading in Edge-driven CAVs Environment

The proposed framework outlines a CAV system wherein vehicles offload tasks and data to nearby infrastructure components such as edge nodes, RSUs, and servers for processing. This setup enables efficient distribution and management of computational workloads between vehicles and infrastructure. The architecture supports V2V and **Vehicle-to-RSU (V2R)** offloading scenarios, facilitating seamless data exchange and real-time communication, as illustrated in Figure 8. By optimizing workload allocation across vehicles, RSUs, edge nodes, and cloud servers, the system enhances resource utilization and overall performance.

## 4 Taxonomy of Task Offloading in CAVs

In this section, we have discussed the task offloading objectives, offloading types and presented the proposed taxonomy of task offloading techniques in CAVs, shown in Figure 9.

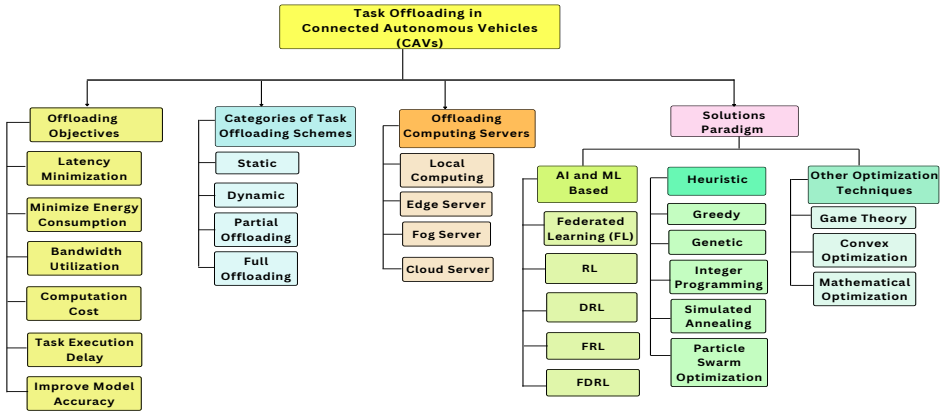



Fig. 9. Taxonomy of task offloading techniques in CAVs.

### 4.1 Task offloading objectives

In the CAVs, offloading objectives aim at maximizing the effectiveness and efficiency of task offloading to edge servers or cloud resources [95]. The primary objectives include:

*4.1.1 Latency.* Minimizing latency is crucial in CAVs for real-time decision-making, such as obstacle detection and navigation. Reducing task execution delays ensures that CAVs respond quickly to dynamic environments, thereby enhancing safety and operational efficiency [7, 129].

*4.1.2 Energy.* CAVs rely on battery-powered systems, making energy efficiency a priority. Offloading computational tasks to external resources helps conserve energy, extending vehicle battery life and supporting the sustainability of autonomous fleets [30]. For example, research in electric CAVs demonstrated that by offloading heavy tasks like HD map updates to edge, energy consumption was reduced by up to 25%, resulting in extended operational hours and lower costs [41, 78].

*4.1.3 Bandwidth.* Optimizing bandwidth usage is necessary for efficient task offloading, especially in environments where network resources are shared. In critical applications, effective bandwidth management ensures that high-priority tasks, such as safety-critical communications, are given precedence without congestion or delay [81].

*4.1.4 Computation Cost.* Minimizing the computational workload on onboard systems by offloading tasks to edge servers can significantly reduce operational costs. Efficient computation offloading can minimize the need for extensive local hardware upgrades and reduce vehicle operational expenses. For instance, in scenarios involving complex AI model inferences, CAVs that offloaded processing to cloud servers reduced on-board computational costs by 20%, thus improving decision-making efficiency while minimizing hardware requirements[13, 91].

*4.1.5 Model Accuracy.* Ensuring high model accuracy in task offloading decisions enhances the reliability of CAV systems. Poor decisions on where and when to offload can lead to incorrect actions, delays, and potential safety risks. For example, a recent analysis of path planning systems in CAVs showed that improving model accuracy by utilizing federated learning-based offloading techniques enhanced real-time route planning accuracy by 18%, reducing traffic congestion and improving overall system efficiency [59, 72].

### 4.2 Offloading Types–Partial vs. Full

Task offloading techniques are categorized into two parts, namely partial and full offloading **[RQ2.1]**. **Partial offloading:-** Partial offloading in CAVs enables the distribution of computing

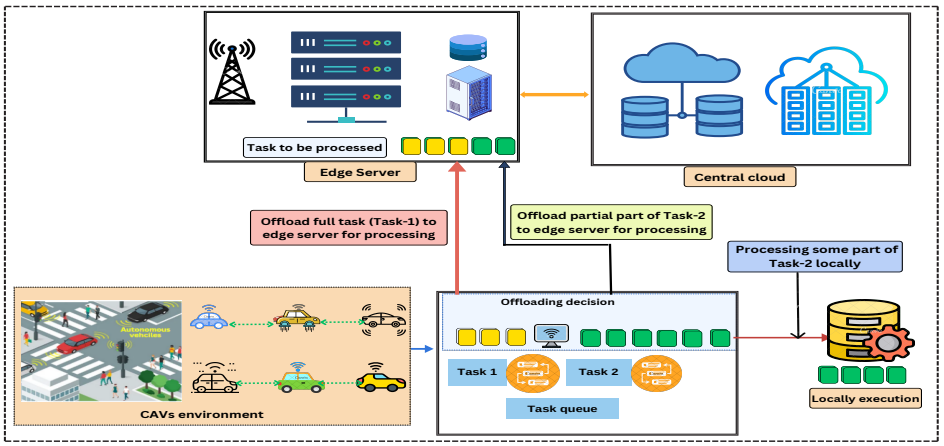



Fig. 10. Illustrating partial and full task offloading technique process.

workloads by handling some tasks locally, such as on edge devices, while outsourcing others to specialized hardware or external systems. This approach is particularly effective for applications with strict deadlines, leveraging parallel processing between the network and end devices. Due to communication network constraints, offloading selected parts of an application, rather than the entire program, enhances performance and efficiency [12].

**Full Offloading:-** Full offloading in CAVs involves offloading the whole computational workload from the vehicle's onboard system to an external edge node or cloud. This perspective is particularly advantageous for resource-intensive jobs, such as high-definition map updates, complex run-time decision-making, and advanced machine learning model inference [105]. Full offloading is essential for applications where local processing is insufficient due to hardware constraints or when achieving lower response times is critical for vehicle safety and performance [58], Figure 10 illustrates a scenario of partial and full task offloading.

## 5 Machine Learning Based Task Offloading

This section presents a comprehensive analysis of **Machine Learning** (**ML**) approaches for task offloading in CAVs, highlighting their capability to support intelligent and adaptive decision-making. It further examines core ML paradigms, FL, RL, and **Deep Reinforcement Learning** (**DRL**) as promising solutions for optimizing offloading strategies in CAV environments.

### 5.1 Introduction to Machine Learning in Task Offloading

ML is revolutionizing task offloading in CAVs by enabling intelligent and adaptive decisions for efficient distribution of computational workloads. ML algorithms can predict network conditions, resource availability, and vehicle status, allowing for the optimal choice of executing tasks near edge devices or remotely on the cloud. The ML uses DRL and **Artificial Neural Network** (**ANN**) models to analyze the real-time data like vehicle location, traffic density, and signal strength to predict the network condition [14, 49]. By integrating ML into task offloading, CAVs achieve superior performance, safety, and reliability, ensuring a seamless and efficient autonomous driving experience. In the following subsection, we have discussed FL and RL as two key subfields of machine learning techniques relevant to task offloading in CAVs **[RQ2.3]**. We further presented proposed taxonomies to comprehensively analyze the existing literature on ML-based task offloading solutions in CAVs systems **[RQ3.1]**.

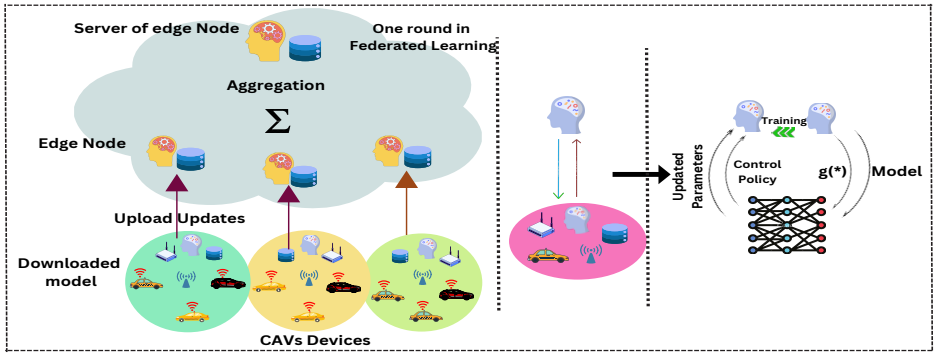



Fig. 11. Federated learning-based task offloading in CAVs system.

### 5.2 Federated Learning-based Task Offloading

FL permits decentralized training of a unified model across multiple edge nodes without requiring centralized data exchange [56, 62]. This approach is ideal for CAVs in task offloading, ensuring data security, low-latency communication, and efficient bandwidth utilization [1, 69]. Its distributed nature supports real-time decision-making and high model accuracy, crucial for autonomous driving systems [31, 111]. For instance, in real-time traffic management, a network of self-driving cars collaboratively forecasts traffic patterns and adjusts routes to reduce travel time and congestion [43, 98].

Figure 11 illustrates the working of an FL-based task offloading system, where CAVs initially train localized models with their information. The locally trained model updates from multiple vehicles are then aggregated at an edge server, improving the vehicles' decision-making capabilities, particularly in route optimization and traffic management. In the following, we have discussed recent studies on FL-based task offloading techniques.

In [81], the authors proposed a federated deep reinforcement learning approach to address the issues of single-agent-based task offloading challenges. Simulations were conducted in a 2.5 × 1.5 km area using PyTorch 1.4.0 and the SUMO simulator. Compared to existing methods like DQN, their proposed FL-DDPG technique outperforms in terms of throughput and queuing delay, relative to GDP and M-DQN models [114].

Compared to [81], in another study [46], the authors have discussed the difficulties of offloading computationally demanding jobs across the vast network of IoT devices in DRL. For each IoT device, the authors proposed the FL method namely **Double Deep Q Learning** (**DDQN**), to minimize the cost of transmission between internet devices and the edge node. The author evaluates the capabilities of the proposed method using settings such as $I$=15 IoT devices and $E$=6 edge nodes. They found that the average usability of FL is higher than that of DRL when additional parameters are taken into account, such as learning rate (0.005), memory size (5000), exploration probability (0.001), and discount rate (0.9).

In [90], the authors address the challenge of information transfer delays in **Vehicular Edge Computing** (**VEC**) due to the nature of automotive tasks. They proposed the FLOR framework, which leverages FL for task offloading and managing the resources to minimize delays. FLOR operates across diverse networks to selecting the optimal network for offloading based on a probabilistic mechanism. Simulations show that FLOR improves offloading efficiency by 20.69% compared to existing methods, ensuring optimal resource allocation.

The increasing number of smart car models has led to latency concerns for data-intensive and latency-sensitive applications. To address these issues, in [101], the authors explored a distributed learning structure for compute offloading in VNs, inspired by FL. Simulations conducted in Matlab,

Table 2. Comparison of Different Federated Learning-based Offloading Techniques

| Author name, Year | Objectives | FL approach | Evaluation methods | Performance metrics | Future directions |
|---|---|---|---|---|---|
| **JIANJI REN *et al.* [46], 2019** | Investigate the task offloading decision of IoT in a supported distributed environment. | Double Deep Q Learning(DDQN). | Experimental setup with edge nodes=6, and Number of IoT devices = 15. | Exploration probability 0.001, memory capacity 5000, learning rate 0.005. | Arrange FL-based learning training at a finer level. |
| **Sahaya Beni *et al.* [90], 2021** | Minimise the delay that occurs in transferring the information of automotive tasks in VEC. | FLOR and e-RRM based Federated Q-Learning. | V2X, Dedicated Short Range, and 6G V2V communication. | Improvement in optimal resource allocation by 20.69%. | Achieve minimal offloading latency and reduced resource cost. |
| **Swapnil *et al.* [101], 2021** | Optimize the virtual network performance in terms of latency and energy consumption. | FL-inspired distributed learning framework. | Matlab with UVs b/w [100-1000], velocity 10m/sec, 80 RSUs. | Jointly minimize the overall latency and the energy consumption. | Select the best RSU for offloading, considering a network of multiple decentralized HAPs. |
| **Bing Lin *et al.* [64], 2021** | Jointly optimize the offloading failure rate and total energy consumption of the offloading process. | Markov decision process (MDP). | The simulation was carried out using Tensor Flow 1.15.0 with Python 3.6. | Reduce failure rate and total energy consumption by 14.2% and 2.5%. | Develop a collaborative offloading strategy in edge cloud architecture to offload complex tasks. |
| **Zhongming ji *et al.* [51], 2021** | Reduce the computational burdens for stragglers in federated learning. | Employs edge-assisted federated learning(EAFL). | The simulation setting, total client k=50, system power p=10w. | Reduce 1.32 in computational delay with respect to FL. | Extend the EAFL for dynamic clients and find optimal reduction in computational delay. |
| **Sungwon Moon *et al.* [81], 2022** | To maximize the throughput of a VEC system under the power constraints of a vehicle. | Federated deep reinforcement learning and DDPG. | The simulation on the SUMO environment in an area of 2.5 × 1.5 km. | Throughput and queuing delay. | Predict the state of the vehicle to increase the performance in terms of power management. |
| **Qixun Zhang *et al.* [127], 2022** | Reduce the task execution delay with different communication and computing constraints. | Federated Reinforcement Learning (FRL). | Simulation bandwidth of RSU and BS is 20 and 40 MHz, power 200 mW. | Reduce delay by 84.6%, 70%, 30.9%, with respect to ERP, FSEP, CORAO. | Enhance V2V and V2I communication. |
| **Chunlin Li *et al.* [59], 2023** | To address the delay and privacy requirements of high-precision map caching in ITS. | Federated deep reinforcement learning (F-DRL). | Simulation environment with Movielens 1m dataset. | Improved training accuracy by 5.55%, avg. backhaul load by 15.17%. | Improving the efficiency of the high-precision map caching problem. |
| **Oguzhan Akyildiz *et al.* [4], 2024** | To optimize resource uses and job processing in CVNs. | RELiOff algorithm. | The simulation was carried out using SUMO with NS-3. | Obtain an efficiency of 99.9% for completed for offloaded tasks . | Investigating advanced algorithms to improve task offloading and resources for CAVs. |

with varying numbers of UVs (99 to 1001). The results demonstrated improved efficiency in terms of latency and energy consumption under these conditions.

The authors of [127] use a **Federated Reinforcement Learning** (**FRL**)-based computation offloading and resource allocation method in the CAVs to address the research problem of lowering the job execution latency with unique communication and computation restrictions. The author's proposed method was evaluated in Python by using the MXNET module. Results show that the given technique effectively minimises the execution delay by 84.6%, 70%, 60.6%, and 30.9%, respectively.

Furthermore, in [59] the authors solved the research challenges to protect privacy, reduce training delay, and improve the accuracy. The authors employ **Federated Deep Reinforcement Learning** (**F-DRL**) and MDP to solve this problem. The author's proposed technique was simulated using a simulation environment with the Movielens 1m dataset. The experimental result shows that the improved training accuracy by 5.55%, average backhaul load by 15.17%, and cache hit by 70%. Table 2 shows the comparison of different federated learning-based offloading techniques.

*5.2.1 Summary of FL-based Task Offloading.* FL-based task offloading for CAVs is an emerging technique that leverages decentralized training in several edge nodes without requiring centralized data exchange. By aggregating locally trained models from multiple vehicles at an edge server, FL enhances decision-making capabilities, such as route optimization and congestion reduction. Various studies have proposed different FL-based frameworks, such as FL-DRL, DDQN, and FRL, to

solve specific problems. So, FL performs better to find the energy consumption, latency reduction, and computational efficiency as well as optimize the route and minimize the congestion. However, federated learning for task offloading faces challenges regarding communication overhead, privacy concerns, asynchronous updates, and inefficiency in environments with unstable network connectivity. Through decentralized task offloading strategy optimization, RL may efficiently address these drawbacks. By utilizing local decision-making and learning optimal rules, this strategy reduces the need for frequent communication.

*5.2.2 Lesson Learned and Findings.* The key insights obtained from applying FL in task offloading for CAVs research are summarized as follows:

(1) To improve efficiency and reduce latency by strategically allocating computing activities between the edge servers and the vehicles.
(2) To identify and permit data to stay on local devices, to avoid the need to transfer sensitive data across the network.
(3) Enabling vehicles to process and analyze real-time data by utilizing local computational resources and collaborative learning from nearby vehicles.
(4) To manage and optimize traffic flow through distributed processing of the massive volumes of data produced by CAVs.

### 5.3 Reinforcement Learning (RL)-based Task Offloading

Due to its capacity to enhance the procedures involved in making decisions and flexibly adapt to changing settings, RL based task offloading is especially beneficial for CAVs. In RL, an agent gains decision-making skills by interacting with the environment and getting feedback in the form of rewards or penalties. Due to the availability of real-time data and experiences, RL-based algorithms are able to enhance their performance continuously [41, 122].

Consider a scenario where a network of CAVs needs to determine whether to assign computing duties to cloud or edge nodes to optimize latency and power consumption while ensuring security and efficiency. The RL agent incrementally identifies the most effective strategies under varying conditions, optimizing for metrics such as minimal latency and power consumption [41]. This adaptive learning capability makes RL-based task offloading significantly more efficient and robust than traditional algorithms [38, 61, 121].

Figure 12 demonstrates the reinforcement learning process, where the agent refines its policy through training and interacting with vehicles in the vehicular domain. In the following, we present state-of-the-art literature on the RL-based task offloading technique.

In [28], the authors employ the Q-learning algorithm and the DDPG algorithm to minimize the delay of task offloading in IoT, and in MEC. The authors' proposed algorithm was simulated using SUMO software and compared with the Best Binary Computation Offloading technique. In comparison to the Best Binary Computation Offloading method, experimental validation reveals that the Q-learning method decreases the latency by 23% and the DDPG method minimizes the delay by 30%.

The authors in [112] addressed the challenge of reducing service latency and network traffic in **multi-access edge computing** (**MEC**) using **Meta Reinforcement Learning** (**MRL**). To solve these issues, the authors proposed an offloading strategy using a **Directed Acyclic Graph** (**DAG**), MRLCO, and a custom **sequence-to-sequence** (**seq2seq**) neural network, implemented in TensorFlow. Their method was evaluated against three baseline approaches heterogeneous earliest finish time, a greedy strategy, and a fine-tuned DRL model. The results of the experiments show that the MRLCO method reduces latency by up to 24% compared to the baseline algorithms.

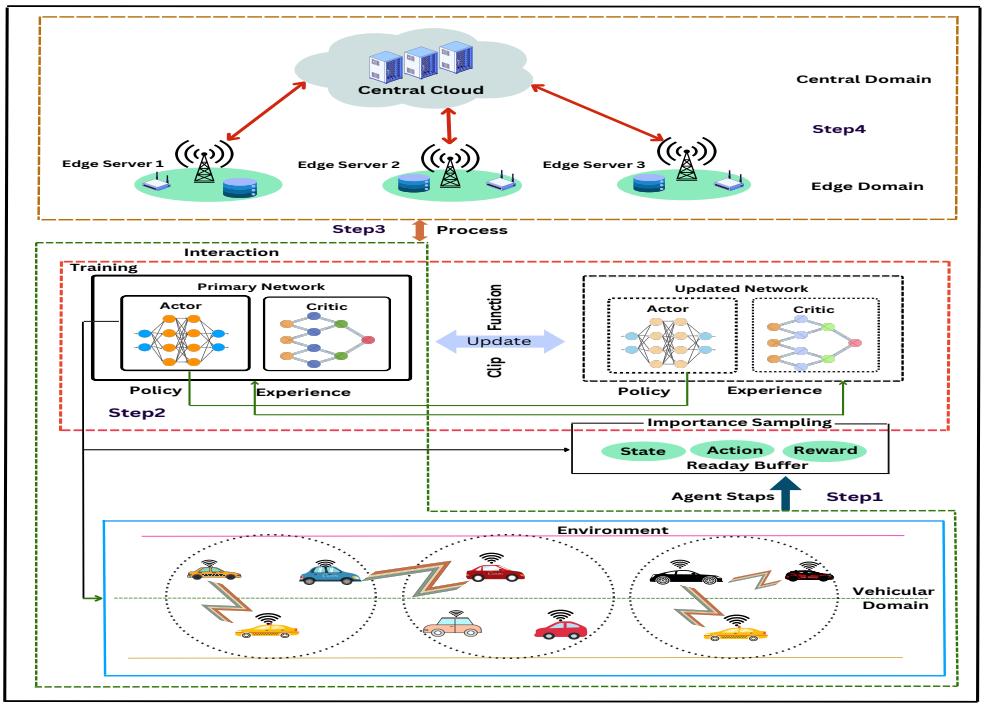



Fig. 12. Reinforcement learning-based task offloading in CAVs system.

Furthermore, to reduce the average processing latency of jobs with vehicle-edge-cloud collaborative computing, the author in [67] uses a DAG and a dependency-aware task offloading method based on **Deep Deterministic Policy Gradient DDPG** (**DA-TODDPG**). Their simulation results indicate that the proposed technique increases performance by an average of 13.62% when compared to the three baseline techniques used by DA-RO, DA-TODQN, and E-TODDPG.

In [34], the authors leverage Q-learning and Deep Q-Network algorithms within a reinforcement learning framework to reduce time and energy overhead in the offloading of computation for **mobile edge computing** (**MEC**). Their approach was implemented using TensorFlow, and evaluated compared to the **Full Offloading and Random Caching** (**FORC**) method. The experimental results show that the proposed technique reduces overall time and energy costs by 26% compared to FORC.

In another study [13], authors addressed the challenge of minimizing tolerance time and ensuring task completion within soft deadlines using **Markov Decision Process** (**MDP**) and value-based reinforcement learning methods. Their proposed approach, tested with SUMO and real bus GPS data, demonstrated superior performance compared to existing RL-based algorithms and greedy strategies, reducing tolerance time by at least 8.81%.

The study in [20] addresses the challenge of enhancing safety by reducing the average collision rates of self-driving cars through a combination of RL and a bespoke collision prediction model. The proposed method, implemented using advanced RL algorithms and a custom simulator, demonstrates a 15% reduction in collision frequency compared to traditional rule-based approaches, while also improving safety and speed.

In [78], the author addresses the challenges of reducing traffic accidents, improving efficiency, and conserving energy for CAVs using **Proximity Policy Optimization** (**PPO**) reinforcement

Table 3. Comparison of Different Reinforcement Learning-based Offloading Techniques

| Author name, Year | Objectives | Approach | Evaluation methods | Performance metrics | Future directions |
|---|---|---|---|---|---|
| **Xinyu Huang *et al.* [48], 2020** | To decrease the energy cost of executing tasks without exceeding the delay constraint in VEC. | MADDPG. | Real-time experimental with vehicles 5 to 13. | Decreases energy consumption by 10% and task completion delay by 4.9%. | Addressing mismatched vehicle speeds and computational resources. |
| **Jin Wang *et al.* [112], 2020** | To reduce network traffic and service latency in multi-access edge computing (MEC). | Meta Reinforcement Learning (MRL). | Tensorflow. | Reduce the latency by up to 25%. | Achieve high Quality-of-Service (QoS) for mobile users and to reduce the network traffic. |
| **Xiaoheng Deng *et al.* [28], 2021** | To reduce the delay of task offloading in IoT for MEC. | Q-learning algorithm and DDPG algorithm. | SUMO using MATLAB. | Reduce the delay. | The algorithm has difficulties due to regional variations in traffic. |
| **Ibrahim A. Elgendy *et al.* [34], 2021** | To reduce the total overhead of time and energy in computation offloading at MEC. | Q-learning and Deep-Q-Network-based algorithm. | TensorFlow. | Reduce time and energy. | Mobility management for mobile users so that the mobile device can be moved dynamically. |
| **Guozhi Liuet al. [67], 2022** | Reducing the average processing delay of tasks with vehicle-edge-cloud collaborative computing. | Directed Acyclic Graph (DAG) and dependency-aware task offloading. | SUMO Simulator. | Reduce processing delay. | Mobility of vehicles and data migration between edge servers during task offloading. |
| **Eduardo Candela *et al.* [20], 2023** | To improve the safety risk by decreasing average collision rates in CAVs. | Bespoke collision prediction model and Reinforcement Learning. | Custom simulator like OMNeT++. | Decreases average collision rates by 15%. | Extend the given framework for functions such as steering control and platooning. |
| **Ta Huu Binh *et al.* [13], 2023** | To minimise the tolerance time and ensure all tasks meet their soft deadlines for quality. | Markov decision process and value-based reinforcement learning. | SUMO with actual bus GPS data. | Reduce the tolerance time and improve quality of service. | To minimise the cost as well as energy for task offloading. |
| **Haitao Min *et al.* [78], 2023** | To reduce traffic accidents and energy saving for CAVs. | Proximity Policy Optimization (PPO), LSTM learning. | NGSIM I-80 dataset. | Saves energy consumption by 9.3–21.9%. | Obtain large amounts of data in very little time for model training. |
| **Geo *et al.* [41], 2024** | Enhance the offloading performance. | Com-DDPG. | SUMO Simulator. | Minimize energy and improve latency. | Analyze the historical log data of edge server. |
| **Srikanta Pradhan *et al.* [89], 2024** | Optimizes resource allocation and task offloading strategies to maximize system utility. | Task offloading and Resource Allocation (TORA) with Double DQN. | SUMO Simulator. | Reduces the number of offloading tasks by 35% and minimize system cost. | Maximizing the offloading of tasks, distribution of resources in a setting with diverse resources. |

learning and trajectory prediction models based on LSTM neural networks. Experiments on the NGSIM I-80 data set show that the proposed approach reduces energy consumption by 9.3 to 21.9%.

To further address the dynamic and often unpredictable conditions encountered during real-time vehicular operations, it is essential to consider the mobility-induced challenges in task offloading. To resolve this, FL offers a promising solution by combining decentralized and centralized training approaches, helping reduce latency, power usage, and bandwidth consumption [81]. FL also supports local model training and small updates, which lowers communication costs and helps maintain network stability [4]**[RQ3.2]**. Moreover, the high mobility of vehicles may cause congestion in hotspot areas. RL can utilize vehicle trajectories to better understand the relationship between vehicle movement and the edge computing environment, improving offloading accuracy and enhancing safety at high speeds. In addition, mobility leads to fluctuating channel conditions, which increases the chances of offloading failures. To address this, multi-agent RL models considering vehicle speed, task type, and delay sensitivity can effectively optimize task offloading and resource allocation, resulting in smarter and more adaptive decisions under dynamic mobility conditions [48]. Table 3 shows the comparison of different reinforcement learning-based offloading techniques.

*5.3.1 Summary of RL-based Task Offloading.* RL-based task offloading for CAVs provides a dynamic and adaptive approach to decision-making by leveraging real-time data and continuous learning. RL allows CAVs to make intelligent decisions on whether to offload computational tasks to cloud or edge nodes to reduce latency and optimize power usage. This is crucial in environments where traditional algorithms struggle due to varying network loads and traffic conditions. Existing

studies have applied RL techniques, including Q-learning, DDPG, and Multi-Agent Deep Reinforcement Learning, to address challenges like task delay, energy consumption, and safety [50, 106]. Despite its potential, real-world deployment of RL-based task offloading remains challenging due to issues such as computational overhead, exploration-exploitation tradeoff, sample inefficiency, long training times, and the need for extensive data, highlighting the need for further research to refine these models for practical applications in CAVs **[RQ3.3]**. To overcome these drawbacks, DRL combines RL with deep learning and deep neural networks to solve complicated and high-dimensional state-action spaces that are inherent in CAVs task offloading. It also improves the capacity to decide on the best work offloading strategy in real-time inside the dynamic surroundings that CAVs operate in, overcoming the restrictions of RL. The functioning of DRL is discussed in Section 5.4.

*5.3.2 Lesson Learned and Findings.* The insights gained from the application of RL in computational offloading within CAVs research are outlined as follows:

(1) To select the optimal offloading decision to balance the load between vehicles and edge.
(2) To perform task scheduling by utilizing techniques like Deep Q-Networks that could reduce offloading failures and energy consumption during computation offloading.
(3) To quickly understand the run-time situation of the traffic and minimize the delay in networks.
(4) To analyze the real-time network states and vehicular movement, to efficiently manage vehicular and infrastructural resources.

### 5.4 Deep Reinforcement Learning (DRL)-based Task Offloading

DRL-based task offloading is particularly advantageous for CAVs since it can handle complex decision-making processes in dynamic and unpredictable situations. DRL, which combines the key characteristic of deep learning with RL, permits the agent to gain knowledge from large-scale data and make complex judgments [113].

Consider a network of CAVs that needs to offload tasks like real-time traffic prediction to edge or cloud servers. Traditional algorithms using static models have concerns about dynamic traffic, network loads, and resource fluctuations [8]. While basic RL may falter with complex data, a DRL-based approach leverages deep neural networks to approximate optimal offloading policies. This enables CAVs to make data-driven decisions that enhance performance metrics such as latency, energy consumption, and reliability [18, 117].

Figure 13, illustrates the working of deep reinforcement learning where the learning agent controls computation offloading between edge nodes and a central server in accordance with service policies and network conditions. To optimize resource utilization and response times, vehicles can have their data processed locally at adjacent edge nodes or offloaded to a remote server. In the following, we have presented a review of the literature on DRL-based task offloading techniques for CAV systems.

The study in [64] proposes a DRL-based computation offloading strategy to jointly optimize failure rate and power consumption. A **Deep Q-Network with Simulated Annealing** (**SA-DQN**) is employed to improve overall performance. Simulations using TensorFlow 1.15.0 and Python 3.6 show a 14.2% reduction in energy usage and a 2.5% decrease in offloading failure rate compared to existing Sarsa and Q-learning methods. The results highlight the need for flexible, end-to-end edge-cloud collaborative computation offloading.

In [73], the authors leverage DRL to optimize task offloading for autonomous vehicles, enhancing environmental perception quality. To achieve superior performance, they employ a MDP to select edge nodes for task execution. The simulation, conducted using SUMO software, considers variables such as **Target Vehicles** (**TVs**), average vehicle speeds, and maximum allowable delay.

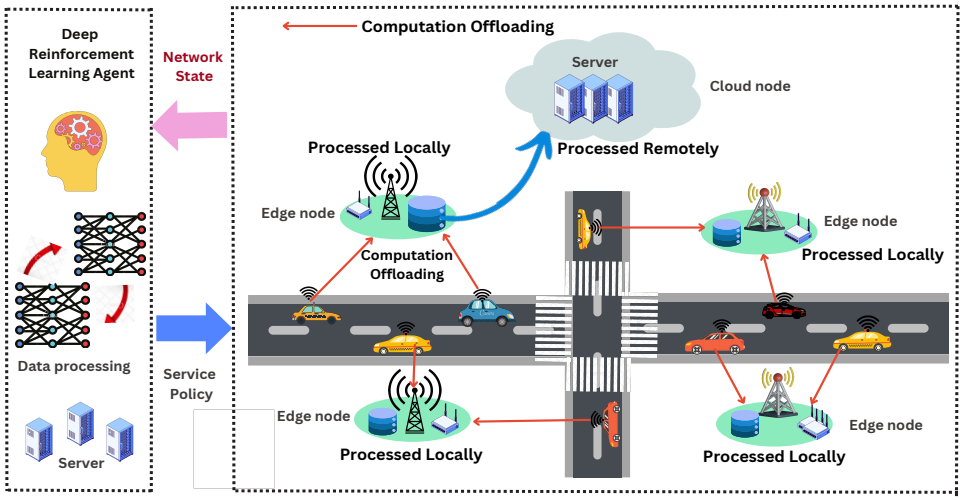



Fig. 13. Deep reinforcement learning-based task offloading in CAVs system.

In [72], the author utilizes a DQN-based DRL architecture with a separate Q-network to optimize task offloading and flying orientation decisions for uncertainty minimization. The proposed method was simulated on a Windows 10 system using PyTorch, CUDA 11.1, and Python. Experimental results show that the average uncertainty was reduced by up to 68%, 52%, 64%, and 77% compared to **Random-Offload** (**RO**), **Random-Trajectory** (**RT**), **Local-Comp-Only** (**LCO**), and **Offload-Comp-Only** (**OCO**) methods, respectively.

In another study [24], authors used **Multi-Agent Deep Deterministic Policy Gradient** (**MADDPG**), a DRL-based technique, to reduce the overall average energy consumption during resource allocation and job offloading in MEC. The SUMO simulator was used to simulate the author's suggested method. According to experimental validation, their proposed method successfully reduces average energy usage in comparison to single-MEC, MASL, Greedy, and Random algorithms by 5%, 75%, 80%, and 90%, respectively.

In [60], the authors aim at optimizing computation offloading in vehicular edge computing by reducing service latency and enhancing reliability. Simulations using VISSIM software show that their method outperforms existing brute-force, random, and greedy strategies, achieving a 22% reduction in average weighted computation cost compared to the greedy approach and a 15% improvement in service time over brute-force techniques.

In [91], the authors address VANET load balancing and job execution costs using a **Multi-task Deep Reinforcement Learning strategy for Scalable Parallel Task Scheduling** (**MDTS**). Simulated with TensorFlow and Python, their method outperforms **Particle Swarm Optimization** (**PSO**) and **Least-Connection Scheduling** (**LCS**), increasing the overall reward from -16.71 and -0.67 to 2.93, and reducing task execution time by 60.2% and 33.3%, respectively.

In [30], the authors employ a DRL approach using the **Deep Deterministic Policy Gradient with Prioritized Experience Replay** (**DDPG-PER**) to effectively reduce energy-delay costs in **Industrial IoT** (**IIoT**) operations. The proposed method, simulated using TensorFlow and Python, is compared against **Random Resource Allocation** (**RRA**), **Uniform Resource Allocation** (**URA**), and standard DDPG techniques. Experimental results show that DDPG-PER reduces energy-delay costs by 27.5% compared to RRA, 16.16% compared to URA, and 1.77% compared to DDPG. Table 4 presents a comparison of various DRL-based offload strategies.

Table 4. Comparison of Different Deep Reinforcement Learning-based Offloading Techniques

| Author name, Year | Objectives | Approach | Evaluation methods | Performance metrics | Future directions |
|---|---|---|---|---|---|
| **Mushu Li *et al.* [60], 2020** | Optimized computation offloading in vehicular edge computing by reducing latency and improving reliability. | Task partition and scheduling algorithm (TPSA) and DDPG. | A row of RSUs, equipped with computing resources, provides communication on the roadside for vehicles. | Improvement 22% in average weighted computing cost and in service time by 15%. | Investigate a multi-agent learning approach to compute the optimal solutions in less information. |
| **Qi Qi *et al.* [91], 2020** | Improves the overall reward, reduces the job completion time, and optimizes the load balancing problem of a VANET. | Multi-task Deep Reinforcement Learning approach. | Use Python, neural networks, and TensorFlow where the number of vehicular nodes is 8 and job children is 10. | Reduced the job execution time by 60.3% and 33.4%. | Learn to reserve certain computing resources for future complicated jobs. |
| **Xing Chen *et al.* [24], 2021** | To minimize the average energy consumption in the resource allocation and task offloading in MEC. | Multi-Agent Deep Deterministic Policy Gradient (MADDPG). | The experimental platform adopts DELL PowerEdge (DELL-R940XA, 4*GOLD-5117). | Reduces the energy consumption by 5%, 75%, 80% with respect to MEC, MASL, Greedy. | Optimized resource allocation and task offloading schemes. |
| **Pin Lv *et al.* [73], 2022** | Improve the environmental perception quality through reasonable task offloading for autonomous vehicles. | Markov Decision Process (MDP). | Simulation on SUMO, parameters 15 TVs on the road with a speed range of [10, 30 m/s]. A vehicle's speed is 15 m/s. | Minimize transmission time, maximize tolerable delay. | Handling environmental challenges and maintaining the regulatory safety standards. |
| **Honghao Gao *et al.* [40], 2022** | To reduce the privacy loss, energy consumption and time delays in optimal task offloading. | Markov Decision Process (MDP). | SUMO using the Reality Mining dataset includes 174 services,703 location points, and 94 users. | Improvement in energy consumption and time delay by 66% and 60% respectively. | To address real-time data for multiuser multiview offloading tasks. |
| **Quyuan Luo *et al.* [72], 2022** | To obtain the optimal task offloading decisions as an uncertainty minimization problem and flying orientation choices. | Deep Q-network (DQN). | Simulate on SUMO, the energy of each UAV is set to 1×104 J, the pre-set flying speed is 5 m/s. | The average uncertainty of the proposed technique has been reduced up to 65.25%. | Consider a practical scenario, the movement of UAVs can be described by pitch angle and velocity. |
| **K Nguyen et al [84],2022** | Online task offloading to parked vehicles during peak business hours. | Meta-heuristic Genetic Algorithm (GA). | Use concert simulator with capacity of parking slop about 50% to 85 % in pick hours . | Accepting the online tasks, improve the average task offloading cost by 40%. | Peak the free time slot and optimise the task offloading during parking time. |
| **HUONG *et al.* [30], 2023** | Minimizes the energy-delay costs associated with tasks in Industrial IoT (IIoT). | Deep deterministic policy gradient with prioritized experience. | Experiment done using Python 3.7, Tkinter 8.6 and Tensorflow 2.6. | To optimize IoT system resources just consider the mobility of devices. | Optimizing IIoT system resources for the mobility of devices. |
| **Pratham oza *et al.* [87], 2024** | Developing a task model for VEC applications that effectively handle both fixed and traffic-dependent deadlines. | Deadline-based task-offloading strategy. | Simulated on VISSIM traffic simulator with Python 3.7. | Increasing the number of tasks assigned to the RSUs and completing all within the deadline. | Extending the suggested model for edge resource optimization across the network. . |

*5.4.1 Summary of DRL-based Task Offloading.* DRL has become a prominent approach for task offloading in CAVs, effectively addressing the challenges of dynamic environments and complex decision-making processes. Existing studies have analyzed various DRL-based strategies, such as **Deep Q-Networks** (**DQN**) and MADDPG, highlighting their effectiveness in optimizing offloading decisions, minimizing latency, and enhancing energy efficiency. Studies show that while DRL improves over RL by handling complex state-action spaces, it still has high computational complexity, training instability, and demands large datasets. These drawbacks reduce its feasibility for real-time applications in CAVS, where quick adaptation and low latency are critical. To overcome these challenges, heuristic algorithms such as **Genetic Algorithms** (**GAs**) offer faster and more resource-efficient alternatives, making them suboptimal solutions compared to DRL for future research to balance performance with computational efficiency in CAV task offloading. So, in the future, the heuristic algorithm could be more useful, like they can use **Ant Colony Optimization** (**ACO**) to optimize task distribution based on traffic, energy constraints, and network conditions. Additionally, by gradually improving offloading tactics, GA would guarantee long-term effectiveness and flexibility in CAVs networks [3, 105].

*5.4.2 Lesson Learned and Findings.* The key findings from the applicability of DRL for task offloading in CAVs research are summarized as follows:

(1) To classify incoming tasks for processing either locally within a vehicle or offloading to edge servers, studies such as [72] and [73] focus on dynamic offloading decisions, utilizing techniques like DRL with DQN and MDP. These works complement each other by enhancing classification strategies based on real-time environmental conditions and vehicle states.
(2) To learn the mobility pattern of vehicles so that optimally assign computational and communication resources among vehicles and edge nodes to maximize efficiency and minimize response time. The research from [60] (DDPG-based) and [24] (MADDPG) reinforces each other by leveraging trajectory and multi-agent data to adaptively allocate resources in motion-sensitive CAV scenarios.
(3) To partition users based on their demand and service requirements, with the goal of enhancing service quality and user experience in CAVs, works such as [91] and [87] focus on addressing various application constraints, including task types and deadlines. Collectively, these studies contribute to a user-aware task partitioning approach and delay-sensitive offloading mechanisms.
(4) To identify the available servers and processors for managing computation offloading tasks, aiming at preventing network congestion and optimize data flow, the works of [30] and [40] focus on optimizing offloading based on server availability and privacy constraints. Their proposed models explored load balancing across cloud and edge servers, in conjunction with scheduling algorithms such as TPSA.

### 5.5 Others ML Technique for CAVs Task Offloading

Table 5 presents a comparative overview of various machine learning techniques beyond FL, RL, and DRL for task offloading in CAVs. It highlights how methods such as supervised learning, unsupervised clustering, meta-learning, and transfer learning contribute to offloading by enhancing model adaptability, learning efficiency, and generalization under dynamic vehicular conditions. Supervised learning approaches enable accurate prediction of offloading decisions using labeled datasets, while unsupervised clustering techniques assist in grouping similar tasks to improve offloading coordination and resource utilization [47]. Meta-learning and transfer learning facilitate rapid adaptation to new environments by leveraging previously learned knowledge, reducing the need for extensive retraining in evolving vehicular networks. The table emphasizes the potential of these methods to optimize latency reduction, energy efficiency, and task execution accuracy, demonstrating their complementary role in achieving reliable and intelligent task offloading within CAV ecosystems.

## 6 Taxonomy of Heuristic-based Task Offloading Algorithms

This section focuses on heuristic-based approaches for task offloading in CAVs, exploring their effectiveness in providing efficient solutions. It reviews various heuristic algorithms that address real-time constraints, resource allocation, and decision-making challenges in CAV systems.

It is designed to address problems more efficiently when conventional techniques are either too slow or incapable of providing an exact solution [27].

Heuristic algorithms play a pivotal role in computational task offloading as they provide efficient mechanisms to distribute workloads between local devices and remote servers, thereby minimizing resource usage and computational overhead [99] **[RQ3.4]**. These methods quickly deliver near-optimal results, reducing energy consumption, computational load, and latency. Leveraging heuristic techniques such as greedy algorithms or evolutionary approaches, systems can dynamically adapt to varying constraints and conditions, ensuring optimal utilization of computational and network

Table 5. Comparison of other ML Techniques used in CAVs Task Offloading

| Author name,Year | Objectives | Approach | Evaluation methods | Performance metrics | Future directions |
|---|---|---|---|---|---|
| **Ziye Liu *et al.* [70], 2024** | To improve road efficiency and reduce energy consumption by vehicle platooning. | Random Forest algorithm with Fuzzy C-Means (FCM) clustering. | The simulation on a highway with a 3+3 lane road, with an average speed of 70 km/h and vehicle densities b/w [100-400]vehicles per km. | This algorithm reduces the mean computation time by 53.2% and path consistency by 9.89%. | Investigating platooning tactics in a range of traffic configurations and environmental circumstances to improve the accuracy. |
| **Yuru Zhang *et al.* [130], 2024** | To minimize the usage of transmission and computing resources under the constraint of end-to-end latency. | EdgeMap and distributed adaptive offloading and resource reservation (DATE) algorithm. | To implement MARL agents, they develop the actor and critic with 3 fully connected layers in PyTorch. | EdgeMap can achieve more than 30% reduction in the total resource usage as compared to the given solutions. | The periodical training of a MARL agent could be challenging at runtime, which requires real-world networks. |
| **Shiyu Fang *et al.* [37], 2025** | To enhance safety, efficiency, and adaptability across various complex scenarios. | Uses CoDriv ingLLM, learnable LLM-driven cooperative driving framework for all-scenario and all-CDA applications. | Simulate the method on highway-env, including single-lane, intersection, roundabout scenario, four-way highway scenario, and merge scenario. | The proposed rule-based, optimization-based, and machine-learning methods in terms of success rate, safety, and efficiency. | Explore the potential applications of CoDriving in other scenarios, including its ability to handle heterogeneous background traffic. |
| **Youcef Djenouri *et al.* [29], 2024** | To identification of vehicular collisions within Internet of Vehicles (IoV) applications. | kNN and Knowledge Guided Deep Learning for Near Crash Detection. | They evaluate the designed system using the Car Crash Dataset(CCD), CrashedCars, and BDD dataset. | KGDL-NCD achieving an AUC (Area Under Curve) metric of 0.95. | Integrating human expertise and knowledge into NCD tasks and Improved Sensor Fusion. |
| **Wenhan Zhang *et al.* [129], 2024** | To improve the offloading latency while the packet loss rate should be low. | Latency Estimation-Based Offloading Algorithm. | Simulate the offloading procedure b/w UE, CN, and EN, where the CN's and EN's computational capacity to 8 × 1010 and to 2 × 1010 CPU cycles/s. | Reduce the end-to-end delay around 60% compared to a random offloading scheme. | Create an offloading technique that lowers the E2E latency by letting a car user choose the cloud or edge server. |

resources, ultimately enhancing user experience and overall system performance [17]. In the following, we have discussed literature on heuristic-based task offloading techniques.

In [115], the author proposed a heuristic-based approach utilizing the **Online Computation Offloading Scheduling and Resource Allocation Algorithm** (**OOSRA**), which integrates a game-theoretic and bin-packing algorithm to arrive at a nearly ideal outcome. Their proposed technique was simulated using the SUMO and OMNeT++ frameworks. Simulation results demonstrate that the Modified First Fit algorithm reduces power consumption by 5.9% compared to the Best Fit method and by 19.59% relative to the Worst Fit method. Additionally, it effectively reduces total computation time by 15% and 45% when compared to OBC and VCC, respectively.

In [84], the author investigates optimal strategies for assigning online computational tasks to **Parked Vehicles** (**PVs**) during peak traffic periods. To address this problem, a dynamic metaheuristic system is employed to adaptively manage varying online demands. The proposed approach, evaluated using a discrete event simulator and compared with existing baseline methods, demonstrates a reduction in average job offloading costs by over 40% and significantly enhances the acceptance rate of online task assignments.

The study in [26] focuses on enhancing system performance and minimizing response time. The authors employ a fast heuristic method for online adaptation and optimize offline scheduling using **Integer Linear Programming** (**ILP**). The proposed methods were evaluated using synthetic task graphs and real-world deployments and compared against the traditional **On-Board Unit** (**OBU**) method. Experimental results show that the online dynamic adaptive scheduling algorithm reduces offloading latency by 34% compared to local OBU execution.

Compared to [26], in [25], the author uses particle swarm optimization in conjunction with a task-offloading decision mechanism to lower total offloading expense and energy usage for MEC-IoV. The SUMO simulator was used to simulate the author's suggested method. The effectiveness

of the suggested approach was compared using the greedy algorithm. The simulation validation demonstrates that the suggested method successfully lowers the overall cost by 14.2%.

In another study [68], the authors aim at reducing the task offloading latency in smart car systems. They have used a brand-new matching-based task offloading technique to address these issues. The SUMO simulator was used to simulate the author's suggested method. The numerical findings showed that, under different real-road topology circumstances, the proposed technique may drastically reduce delays and duplicate the whole vehicle motion using suitable mobility models [83].

In [11], the authors applied the **Cost-effective Heuristic Algorithm for Task Offloading** (**CHAT**) to optimize the cost function by reducing computational time, delays, and overall server energy usage. The task offloading decision is framed as a non-linear integer optimization problem, targeting the minimization of operator expenses (CAPEX/OPEX) while meeting delay constraints. The cost function incorporates server computational load and energy consumption, with delays modelled using an M/G/1 queuing system. The problem is solved using ILP for optimal results, and a greedy heuristic approach (CHAT) for reduced computational complexity. The proposed method, tested using the SUMO simulator, outperforms baseline ILP techniques, achieving a 35% reduction in overall costs.

In [108], the study aims at reducing task response times to improve user service quality. The author employs the KMM algorithm, a greedy server selection approach, and a greedy-based matching algorithm under dynamic conditions. The proposed method, simulated using the SUMO simulator on MATLAB, outperforms the **Two-Stage Optimal Pathfinding and Mapping** (**TOPM**) algorithm by increasing the job offloading rate by 4.9% and reducing RSU utilization by 44.98%. Table 6 presents a summary of various heuristic-based task offloading techniques.

### 6.1 Summary of Heuristic-based Task Offloading

Heuristic-based task offloading techniques in CAVs provide efficient mechanisms for distributing computational tasks between local devices and remote servers, minimizing latency, energy consumption, and computational overhead [104]. These algorithms, such as ACO, SA, and GA, dynamically adapt to the changing environment and constraints of CAV systems, enabling near-optimal performance with lower computational complexity compared to more intensive methods like DRL [71, 75]. Studies have shown that its significantly enhance task offloading efficiency, reduce response times, and enhance the overall performance of the system, making them suitable for run-time vehicular applications [32, 94]. But still, there are some limitations of these algorithms, like suboptimal solutions, lack of adaptability, problem specificity, and limited exploration of the solution space. To further enhance resource usage and service quality in dynamic vehicular networks, future research should concentrate on combining these heuristics with more complex decision-making frameworks.

*6.1.1 Lesson Learned and Findings.* The key insights findings for the applicability of heuristic algorithms in task offloading for CAVs research are listed below:

(1) Heuristic methods can be explored to optimize the task execution sequence and timing to balance computing load across available resources, thereby preventing delays and congestion.
(2) Heuristic-based solutions can offload strategies to enhance system performance in response to changing factors such as server load, network quality, and vehicle speed.
(3) Heuristics can be used to ensure reliable data transmission between CAVs and offloading nodes by managing and optimizing network capacity and connectivity in real-time.
(4) To allocate computational resources that could provide the optimal combination of local, edge, and cloud servers, minimising processing time and energy consumption.

Table 6. Comparison of Different Heuristic-based Task Offloading Techniques

| Author name, Year | Objectives | Heuristic approach | Evaluation methods | Performance metrics | Potential directions |
|---|---|---|---|---|---|
| **PENGJU LIU *et al.* [68], 2019** | How to reduce latency of job offloading in edge computing for vehicles. | Matching-based task offloading algorithm. | Simulation on SUMO Simulator. | Minimizing the total offloading delay. | Determine the task offloading strategy under various constraints. |
| **ZHEN WANG *et al.* [115], 2020** | How to minimize the total use time and reducing the energy consumption. | Game-theoretic online algorithm. | Simulated on SUMO and OMNeT++. | Procession time and energy consumption | Minimize the total simulation time. |
| **Mingyue Cui *et al.* [26], 2020** | Achieving optimal system performance while reducing response time. | Integer linear programming (ILP),ODA. | Experimental. | Response time, system performance. | Model the network instability and expand approach to multivehicle. |
| **JIANAN SUN *et al.* [107], 2020** | Efficiently optimizes the task delay and computing resource consumption. | joint offloading decision and task scheduling algorithm (JODTS). | Simulation using an NS-3 simulator. | Execution cost. | Expand the JODTS to manage loading computation where the tasks and VEC n/w are ambiguous. |
| **KhoaNguyen *et al.* [83], 2020** | To efficiently improve online heterogeneous task scheduling. | Min-max heuristic algorithm. | Discrete event simulator. | Task acceptance ratios. | Optimal utilization of computing resources of parked vehicles. |
| **ANDREY BELOGAEV *et al.* [11], 2020** | To find the queuing delays in Intelligent Transportation Systems (ITS). | Cost-effective Heuristic Algorithm for Task offloading (CHAT). | Simulation on SUMO. | Minimize the computational time, delays, and the total server energy. | Identify the other cost functions, both non-linear and linear. |
| **Shujuan Tian *et al.* [108], 2021** | Improving the user's service quality. | Greedy-based matching algorithm under dynamic conditions (GMDC). | Simulation on SUMO using MATLAB. | Minimize the response time of the tasks. | Study task offloading decisions under the condition of multiple intersections. |
| **Jun Cheng *et al.* [25], 2021** | Reduce the total offloading cost and energy consumption for MEC-IoV. | Particle swarm optimization. | Simulation on SUMO. | Total offloading cost and energy consumption. | Classifying tasks, such as QoS constraints, CPU requirements, and priority. |
| **KHOA NGUYEN *et al.* [84], 2022** | How to efficiently offload online computational tasks to parked vehicles (PVs) during peak business hours. | Meta-heuristic algorithm. | Discrete event simulator. | Accepting the online tasks and improves average task offloading cost by at least 40%. | Making a proper decision on which offloading strategies are selected to optimize the generated revenues. |
| **Zhu Xiao *et al.* [118], 2023** | Minimize the execution delayand safety in autonomous driving system | Two-layer binary intelligent firefly with Heuristic algorithm. | Simulation on SUMO and MATLAB. | Enhancing the safety, efficiency and resource utilization. | Improved the resource efficiency of the autonomous driving system. |
| **Thilo Strauss *et al.* [94], 2024** | To determine which vehicle offloads computation to which computer resource in the city. | Differential Orchestration Method (DOM). | Experimental. | Increase the processing time. | Coordination of wireless transmissions for vehicular ad-hoc communication networks. |

### 6.2 Meta-analysis of Existing ML and Heuristic Task Offloading Techniques

We conducted an in-depth technical analysis of the key methodologies employed in ML and heuristic-based approaches for task offloading in CAVs. We provide a comparative analysis of around 55 studies, employing FL 14, RL 13, DRL 17 and heuristics methods 11. These corresponding studies are also summarized from Tables 2 to 6 for transparency and reproducibility. We present the overall findings in Figure 14(b), where DRL techniques dominate the literature, accounting for 28.3%, with their methods like DDPG, PPO, and DQN, and so on. FL methods follow at 26.7%, with key techniques like FedAvg, FedProx, and MOON contributing almost equally. Same as RL, and Heuristic methods also play an important role in CAV task offloading. These methods target key performance metrics such as latency, energy, and bandwidth optimization, shown in Figure 14(a). A quantitative comparison in Figure 15 further illustrates the reductions in these metrics. FLOR [90] achieves the highest reduction in task completion delay (70%), with DDQN [46] showing moderate improvements across all metrics. MRL notably reduces latency, while Com-DDPG [41] excels in energy reduction, highlighting the efficiency of different offloading techniques.

## 7 Taxonomy of Task Offloading Evaluation Environment, Metrics, and Datasets Available

This section provides a comprehensive taxonomy of the execution environments used for task offloading, the performance metrics employed for evaluating job offloading strategies, and the datasets available for research in the CAV system.

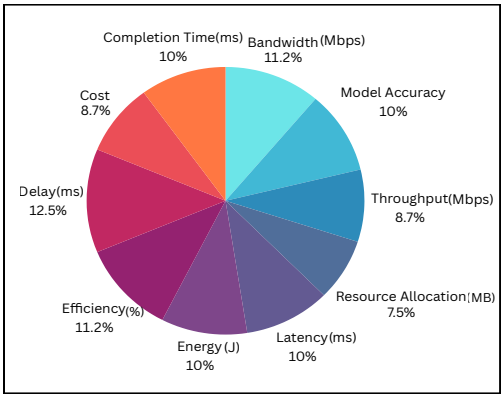



Fig. 14(a) Meta analysis of prior results.

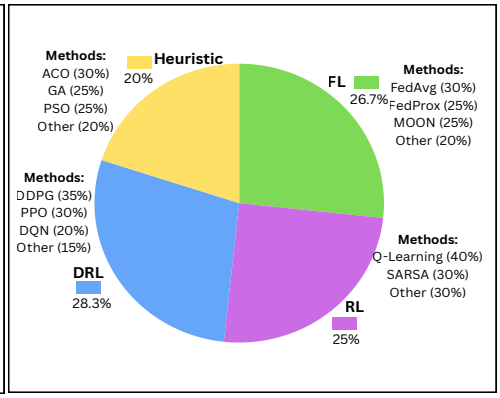



Fig. 14(b) Deep analysis of key methodology.

Fig. 14. Meta-analysis of performance metrics and offloading methods in CAVs.

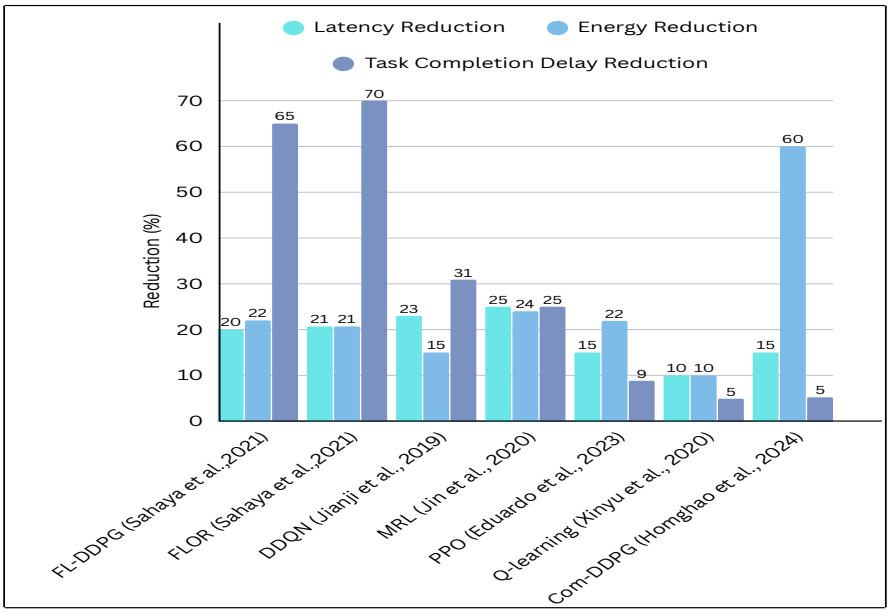



Fig. 15. Quantitative comparison of task offloading techniques in CAVs.

### 7.1 Taxonomy of Task Offloading Evaluation Environment

This subsection presented a taxonomy of evaluation environments for task offloading in CAVs. This taxonomy categorises the available evaluation methods, including simulators, testbeds, and machine learning frameworks, which are essential for implementing task-offloading environments in CAVs **[RQ4.2]**. Simulators such as SUMO and NS-3 provide virtual environments for testing and modelling vehicular scenarios. At the same time, testbeds like NVIDIA GPUs offer physical hardware for real-time experimentation. Additionally, ML frameworks like TensorFlow and PyTorch facilitate developing and testing algorithms designed to optimize offloading strategies. Figure 16 shows the

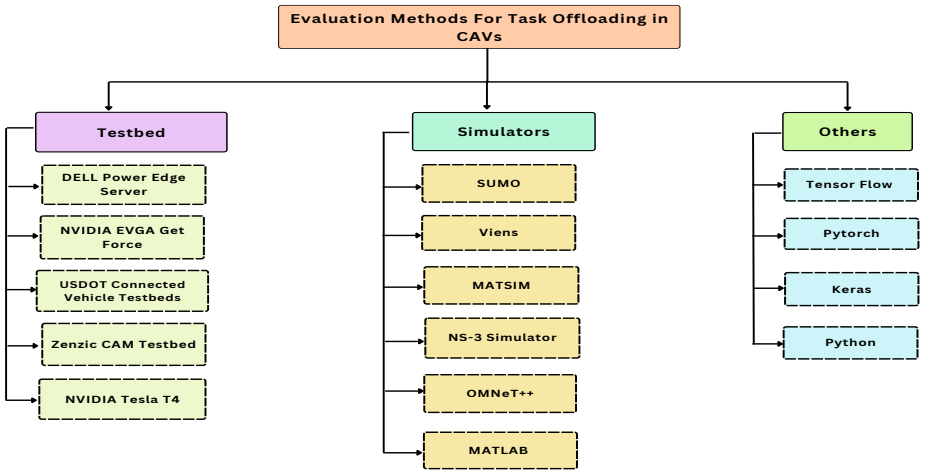



Fig. 16. Taxonomy of evaluation methods for task offloading in CAVs.

Table 7. Classification of Task Offloading Execution Environment

| Execution Env. | Main Focus Type | Characteristics | Advantages | Limitations | Applications |
|---|---|---|---|---|---|
| SUMO | Traffic and Mobility Simulation. | Supports multi-modal transport (cars, buses, etc.). | Supports real-time simulation. | Limited in wireless communication modeling | Traffic planning, congestion management. |
| Veins | Vehicular Networking Simulation. | Integrates NS-3 with OMNeT++, supports vehicular ad-hoc networks. | Realistic vehicle-to-vehicle communication. | Requires both NS-3 and OMNeT++ setup, complex integration. | Vehicle simulation, V2X communication, ITS. |
| MATSim | Multi-agent Transportation System Simulation. | Open-source, agent-based modelling, large-scale transportation simulation. | Customizable, handles large transportation models, simulation in real. | High computational requirements, not suitable for real-time simulation. | Urban planning, transportation policy evaluation. |
| NS-3 | Network Simulation | Open-source, focuses on network simulation, supports networking protocols. | Detailed network modelling, widely used in academia, is highly extensible. | Limited mobility and vehicle simulation, steeper learning curve. | Wireless network research, IoT, 5G, VANETs. |
| OMNeT++ | Network Simulation and Modelling. | Discrete event simulation, modular architecture, supports various protocols. | Supports various modules and protocols, strong community. | Complex setup, requires understanding of simulation architecture. | Network protocol design, wireless communication, IoT, vehicular networks. |
| MATLAB | General Simulation and Modelling. | Commercial tool, supports mathematical modelling, algorithm development. | Highly versatile, extensive built-in functions, great for algorithm testing. | Expensive, not as specialized for network or vehicular simulation. | Engineering, signal processing, control systems, and algorithm development. |

taxonomy of the task offloading evaluation environment. Further, Table 7 shows the characteristics, advantages, limitations and application scenarios of different execution environments.

### 7.2 Taxonomy of Performance Metrics

Performance metrics are vital for evaluating task offloading in CAVs, covering aspects like task accuracy, latency, cost, and energy consumption. Additionally, other metrics, like scalability and system utility maximization, ensure that offloading solutions can adapt to changing environments and maintain optimal performance under diverse conditions. This taxonomy is a comprehensive

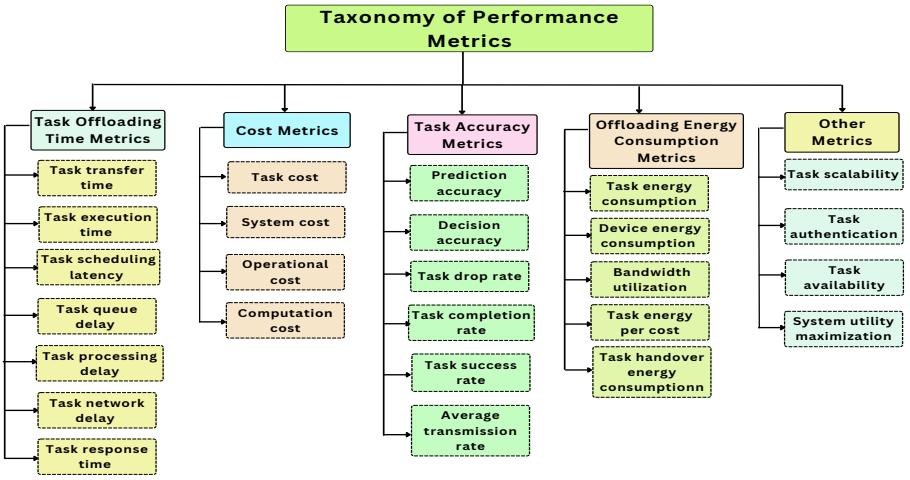



Fig. 17. Taxonomy of performance metrics.

Table 8. Summary of Benchmark Datasets used in CAVs

| Dataset Category | Description | Characteristics | Applications |
|---|---|---|---|
| KITTI dataset [63] | Data for 3D object detection, visual odometry. | Focus on autonomous driving. | Autonomous driving, Object detection, planning, semantic segmentation. |
| Waymo open dataset [35] | Use in 3D object detection, tracking. | Large-scale dataset. | Autonomous vehicle perception, 3D object detection, Lane, motion detection. |
| Berkeley Deep Drive [53] | Use in object detection, lane detection. | 100K images, includes various driving scenarios (urban, rural, highways). | Autonomous driving, Computer vision research, Traffic analysis. |
| Movielens [59] | It includes user ratings and movie metadata. | Contains movie ratings data, Users rate movies on a scale of 1-5. | Recommender systems, Collaborative filtering, Movie recommendation algo. |
| DLT-2 dataset [39] | Use for building and monitoring data pipelines. | Traffic related data (2D/3D vehicle detection), Video recordings and LiDAR data, High annotation quality. | Autonomous vehicle perception - Vehicle detection - Traffic flow analysis. |
| Chicago street dataset [44] | Data on the street center lines in Chicago. | High-resolution imagery, 2D/3D vehicle detection, Ground-truth annotations for traffic scene understanding. | Traffic analysis, Autonomous vehicle perception, City infrastructure monitoring. |
| Apollo Scape dataset [120] | Offers pixel-wise semantic annotation in 2D. | Diverse urban and highway driving data and lane detection. | Autonomous driving, 3D object detection, Semantic segmentation, Traffic sign recognition. |
| C3PW [22] | Use in human pose estimation and forecasting. | Pedestrian-centric dataset, Focuses on pedestrian detection. | Safety analysis in autonomous driving, Motion tracking and forecasting. |

framework for systematically analyzing and improving task offloading in CAVs networks. Figure 17 shows the taxonomy of performance metrics **[RQ4.1]**.

### 7.3 Dataset for Connected Autonomous Vehicles

Datasets play a crucial role in CAV research. Common datasets such as the KITTI and Waymo Open datasets are used for jobs like 3D object recognition, visual odometry, and tracking. The **Berkeley Deep Drive** (**BDD**100K) dataset focuses on object detection and lane detection, while the ApolloScape dataset offers detailed pixel-wise semantic annotations for 2D visual understanding. Table 8 shows the datasets used in CAVs **[RQ4.3]**. The selection of appropriate datasets directly affects the validity, scalability, and real-world applicability of task offloading research. Furthermore, it critically impacts task offloading research by ensuring realistic scenario modelling, enabling accurate ML training, supporting performance benchmarking, and enhancing generalizability across diverse CAVs environments.

Leveraging large-scale real traffic datasets to train and evaluate task offloading models significantly enhances their effectiveness by capturing the complexity of real traffic conditions. Such datasets provide diverse scenarios, including varying traffic densities, road types, vehicle behaviours, and environmental factors that help models learn robust offloading strategies adaptable to dynamic and unpredictable situations. This improves accuracy, reduces latency, and optimizes resource allocation in practical, complex traffic environments, making the models more reliable and scalable for real deployment in CAVs.

## 8 Challenges and Future Opportunities

Through an extensive literature review, we have identified several open issues and challenges that require further exploration in future research on task offloading for connected autonomous vehicle systems. A detailed discussion of each of these issues is highlighted in the following **[RQ4.4]**.

**1. Network Latency and Reliability:** It is a critical concern for CAVs that require task offloading, as high latency can delay crucial decisions such as braking or turning, potentially compromising safety [126]. Unstable networks and frequent disruptions can lead to task failures or incomplete data transmission, undermining real-time processing and vehicle coordination. Therefore, further research is needed on technologies like multi-access edge computing, adaptive routing and load balancing, and collision warning that can minimize latency and enhance the reliability of task offloading in CAVs environments [16].

**2. Task offloading Energy Consumption:** Another significant challenge associated with task sharing in CAVs is the high energy consumption, as substantial computing power is required whether tasks are processed on the vehicle or offloaded to nearby edge nodes. CAVs must also balance computational capabilities with battery life, making energy optimization crucial for overall performance and vehicle longevity. This issue can be addressed by employing low-power communication techniques and developing energy-efficient offloading strategies like energy-aware task offloading, and energy-efficient scheduling algorithms. Therefore, further research is needed to explore these advanced energy minimization techniques.

**3. Heterogeneous Network Environments:** Heterogeneous network environments, characterized by varying connectivity standards, bandwidth, and latency, such as 6G and Wi-Fi, pose significant challenges for task offloading in CAVs. This variability complicates the rapid decision-making required for effective offloading, increased energy consumption, and reduced reliability in dynamic vehicular settings. To address this issue there is a need of further studies that could apply the advanced techniques like network slicing, software-defined networking, and so on.

**4. Real-Time Analytics:** Various ML-based approaches, including FL, RL, and DRL, and so on, can be employed to predict ideal offloading strategies in real time. Instantaneous decision-making is essential in CAVs' task offloading scenarios [3]. To enhance this capability, integrating FL models with 6G networks and edge computing platforms, such as NVIDIA Jetson and Intel OpenVINO, provides computational power for handling complex real-time analytics. Consequently, further research is needed on this collaborative model to strengthen real-time analytics capabilities [110].

**5. Enhanced Vehicle-to-Everything (V2X) Communication:** To enhance communication in CAVs task offloading environments, technology like 5G, 6G, **Software Defined Networking (SDN)**, and adaptive traffic management are employed. Additionally, combining 6G technology with AI and ML could be instrumental in advancing V2X communication for CAV task offloading [123]. Therefore, further research is required on ML-based models paired with improved communication technology to improve overall communication efficiency [19, 119].

**6. Collaborative Task Offloading:** In order to maximize performance, minimize latency, and boost efficiency, collaborative task offloading in CAVs dynamically distributes computing duties among neighboring vehicles, edge servers, and cloud resources. This method addresses issues including network stability and security while enhancing scalability, dependability, and energy efficiency. Future research could benefit from exploring blockchain, 6G, and AI-integrated solutions to further enhance the effectiveness of collaborative offloading [9, 73].

**7. Resource Discovery and Management:** One of the main challenges in CAVs is the effective detection and management of computer resources in vehicles. To collect data on available resources from other vehicles, a nearby vehicle is typically assigned as the leader of the vehicular cloud. After that, the head vehicle is responsible for delegating responsibilities and overseeing the vehicular cloud's entire resource pool. In order to save costs and boost productivity, it is crucial to investigate vehicular edge computing offloading solutions and use some advanced techniques like distributed hash tables, FL, and game theory-based resource allocation methods [52, 79].

**8. Parking:** Parked vehicles can offload computational tasks to edge or cloud infrastructure via edge computing. FL technique can be used to collaboratively train models across multiple parked vehicles without transferring sensitive data, improving the overall system while maintaining privacy [93].

**9. Steering Control:** Steering control in CAVs involves processing sensor data to control vehicle movement and make real-time adjustments to navigate through traffic. **Model Predictive Control** (**MPC**) is a widely used technique for steering control in autonomous vehicles. It predicts the next states of the vehicle and adjusts the steering based on the vehicle's model, optimizing for smooth and stable control. Task offloading for steering control can be done by leveraging edge computing, where real-time processing can be done closer to the vehicle to reduce latency.

**10. Platooning:** Platooning is a coordinated group of CAVs travelling closely to enhance traffic flow, fuel efficiency, and safety using synchronised communication. Challenges include ensuring low-latency communication, real-time coordination, resource limitations, and safety under dynamic conditions. The RL algorithms can be used to dynamically determine the best task offloading tactic based on the current platoon configuration and the surrounding environment, optimizing communication and computational efficiency. This enables adaptive resource management, real-time decisions, and improved platoon stability and safety [55].

**11. Addressing Tradeoffs in Task Offloading**: While ML-based task offloading methods offer significant performance improvements, they face challenges in generalization and computational complexity. A significant challenge with ML-based task offloading strategies is their reliance on historical data for training models. These models may not generalize well to unseen scenarios, particularly in dynamic environments like CAVs, where real-time data varies significantly. Future work should focus on developing adaptive algorithms or hybrid approaches combining ML and traditional optimization to enhance model robustness. On the other hand, optimization-based offloading techniques, such as game-theoretic algorithms or heuristic-based methods, are often computationally intensive, especially when expanding to large networks with multiple vehicles and edge servers. These tradeoffs should be carefully considered when selecting offloading strategies for CAVs, as optimizing for one aspect (e.g., accuracy or computational cost) may lead to suboptimal performance in other areas, such as latency or generalization.

## 9 Concluding Remarks

CAVs have transformed the transportation industry by enabling advanced traffic management, enhancing quality of life, and promoting road safety, thereby reducing human errors that frequently lead to accidents. However, as CAVs advanced, their limited onboard computing power, storage, and

energy resources posed challenges in meeting the demands of applications requiring real-time data processing, autonomous navigation, and adaptive cruise control. Task offloading played a crucial role in the development and deployment of CAVs, enhancing operational efficiency and safety. This study presented a comprehensive and critical survey of task offloading mechanisms for CAVs. We presented a high-level architectural diagram of CAV systems and proposed a layered taxonomy that encapsulated task offloading objectives, execution environments, and offloading strategies. Our review analyzed a wide array of offloading algorithms, including ML, FL, RL, DRL, and heuristic-based approaches, while also evaluating key performance metrics and simulation environments. Through this work, we identified current trends, comparative insights, and performance gaps in task offloading research specific to CAVs, offering a consolidated resource for researchers and practitioners.

Future research can focus on hybrid offloading strategies combining ML and heuristic methods for dynamic CAV environments. Exploring 6G, quantum computing, and multi-agent cooperation can further enhance efficiency. Real-world testing and digital twin simulations will also be vital for practical deployment.

## References


[1] Sawsan AbdulRahman, Hanine Tout, Hakima Ould-Slimane, Azzam Mourad, Chamseddine Talhi, and Mohsen Guizani. 2020. A survey on federated learning: The journey from centralized to distributed on-site learning and beyond. *IEEE Internet of Things Journal* 8, 7 (2020), 5476–5497.

[2] Hafiz Usman Ahmed, Ying Huang, Pan Lu, and Raj Bridgelall. 2022. Technology developments and impacts of connected and autonomous vehicles: An overview. *Smart Cities* 5, 1 (2022), 382–404.

[3] Manzoor Ahmed, Salman Raza, Muhammad Ayzed Mirza, Abdul Aziz, Manzoor Ahmed Khan, Wali Ullah Khan, Jianbo Li, and Zhu Han. 2022. A survey on vehicular task offloading: Classification, issues, and challenges. *Journal of King Saud University-Computer and Information Sciences* 34, 7 (2022), 4135–4162.

[4] Oğuzhan Akyıldız, Feyza Yıldırım Okay, İbrahim Kök, and Suat Özdemir. 2024. Road to efficiency: Mobility-driven joint task offloading and resource utilization protocol for connected vehicle networks. *Future Generation Computer Systems* 156 (2024), 157–167.

[5] Julio C. S. Dos Anjos, Kassiano J. Matteussi, Fernanda C. Orlandi, Jorge L. V. Barbosa, Jorge Sá Silva, Luiz F. Bittencourt, and Cláudio F. R. Geyer. 2023. A survey on collaborative learning for intelligent autonomous systems. *Computing Surveys* 56, 4 (2023), 1–37.

[6] Sriram Ashokkumar, Anirudh Jayendra, Sam Tobin, Ariel Leykin, Robert Stegeman, Abhiraj Dashora, Bryan Look, Joseph Koenig, Brian Hu, Mason Crooks, et al. 2024. *Rapid Development of an Autonomous Vehicle for the SAE AutoDrive Challenge II Competition*. Technical Report. SAE Technical Paper.

[7] Anjum Mohd Aslam, Aditya Bhardwaj, and Rajat Chaudhary. 2025. Quantum-resilient blockchain-enabled secure communication framework for connected autonomous vehicles using post-quantum cryptography. *Vehicular Communications* 52 (2025), 100880.

[8] HM Abdul Aziz and Sanjoy Das. 2023. Deep reinforcement learning applications in connected-automated transportation systems. In *Proceedings of the Deep Learning and Its Applications for Vehicle Networks*. CRC Press, 133–164.

[9] Bakary Badjie, José Cecílio, and Antonio Casimiro. 2024. Adversarial attacks and countermeasures on image classification-based deep learning models in autonomous driving systems: A systematic review. *Computing Surveys* 57, 1 (2024), 1–52.

[10] Rasmeet S. Bali, Neeraj Kumar, and Joel J. P. C. Rodrigues. 2014. Clustering in vehicular ad hoc networks: Taxonomy, challenges and solutions. *Vehicular Communications* 1, 3 (2014), 134–152.

[11] Andrey Belogaev, Alexey Elokhin, Artem Krasilov, Evgeny Khorov, and Ian F. Akyildiz. 2020. Cost-effective V2X task offloading in MEC-assisted intelligent transportation systems. *IEEE Access* 8 (2020), 169010–169023.

[12] Suzhi Bi and Ying Jun Zhang. 2018. Computation rate maximization for wireless powered mobile-edge computing with binary computation offloading. *IEEE Transactions on Wireless Communications* 17, 6 (2018), 4177–4190.

[13] Ta Huu Binh, Hiep Vo, Binh Minh Nguyen, and Huynh Thi Thanh Binh. 2023. Reinforcement learning for optimizing delay-sensitive task offloading in vehicular edge-cloud computing. *IEEE Internet of Things Journal* 11, 2 (2023), 2058–2069.

[14] Christian Birchler, Sajad Khatiri, Bill Bosshard, Alessio Gambi, and Sebastiano Panichella. 2023. Machine learning-based test selection for simulation-based testing of self-driving cars software. *Empirical Software Engineering* 28, 3 (2023), 71.

[15] Archishman Biswas, M. A. Obayed Reon, Prangon Das, Zinat Tasneem, S. M. Muyeen, Sajal K. Das, Faisal R. Badal, Subrata Kumar Sarker, Md Mehedi Hassan, Sarafat Hussain Abhi, et al. 2022. State-of-the-art review on recent advancements on lateral control of autonomous vehicles. *IEEE Access* 10 (2022), 114759–114786.
[16] Azzedine Boukerche and Victor Soto. 2020. Computation offloading and retrieval for vehicular edge computing. *ACM Computing Surveys* 6 (2020).
[17] Bouziane Brik, Junaid Ahmed Khan, Yacine Ghamri-Doudane, Nasreddine Lagraa, and Abderrahmane Lakas. 2022. GSS-VF: A game-theoretic approach for service discovery in fog network of vehicles. *IEEE Transactions on Green Communications and Networking* 6, 3 (2022), 1418–1427.
[18] Ishan Budhiraja, Neeraj Kumar, Himanshu Sharma, Mohamed Elhoseny, Yahya Lakys, and Joel J. P. C. Rodrigues. 2022. Latency-energy tradeoff in connected autonomous vehicles: A deep reinforcement learning scheme. *IEEE Transactions on Intelligent Transportation Systems* 24, 11 (2022), 13296–13308.
[19] Rajkumar Buyya, Satish Narayana Srirama, Giuliano Casale, Rodrigo Calheiros, Yogesh Simmhan, Blesson Varghese, Erol Gelenbe, Bahman Javadi, Luis Miguel Vaquero, Marco A. S. Netto, et al. 2018. A manifesto for future generation cloud computing: Research directions for the next decade. *ACM Computing Surveys* 51, 5 (2018), 1–38.
[20] Eduardo Candela, Olivier Doustaly, Leandro Parada, Felix Feng, Yiannis Demiris, and Panagiotis Angeloudis. 2023. Risk-aware controller for autonomous vehicles using model-based collision prediction and reinforcement learning. *Artificial Intelligence* 320 (2023), 103923.
[21] Vishnu Pandi Chellapandi, Liangqi Yuan, Christopher G. Brinton, Stanislaw H. Żak, and Ziran Wang. 2024. Federated learning for connected and automated vehicles: A survey of existing approaches and challenges. *IEEE Transactions on Intelligent Vehicles* 9, 1 (2024), 119–137. DOI: 10.1109/TIV.2023.3332675
[22] Hanlin Chen. 2021. *Adaptive Safety and Cyber Security for Connected and Automated Vehicle System*. Ph. D. Dissertation. Purdue University Graduate School.
[23] Min Chen and Yixue Hao. 2018. Task offloading for mobile edge computing in software defined ultra-dense network. *IEEE Journal on Selected Areas in Communications* 36, 3 (2018), 587–597.
[24] Xing Chen and Guizhong Liu. 2021. Energy-efficient task offloading and resource allocation via deep reinforcement learning for augmented reality in mobile edge networks. *IEEE Internet of Things Journal* 8, 13 (2021), 10843–10856.
[25] Jun Cheng and Dejun Guan. 2021. Research on task-offloading decision mechanism in mobile edge computing-based internet of vehicle. *EURASIP Journal on Wireless Communications and Networking* 2021, 1 (2021), 101.
[26] Mingyue Cui, Shipeng Zhong, Boyang Li, Xu Chen, and Kai Huang. 2020. Offloading autonomous driving services via edge computing. *IEEE Internet of Things Journal* 7, 10 (2020), 10535–10547.
[27] Taiping Cui, Yuyu Hu, Bin Shen, and Qianbin Chen. 2019. Task offloading based on lyapunov optimization for mec-assisted vehicular platooning networks. *Sensors* 19, 22 (2019), 4974.
[28] Xiaoheng Deng, Jian Yin, Peiyuan Guan, Neal N. Xiong, Lan Zhang, and Shahid Mumtaz. 2021. Intelligent delay-aware partial computing task offloading for multi-user industrial Internet of Things through edge computing. *IEEE Internet of Things Journal* 10, 4 (2021), 2954–2966.
[29] Youcef Djenouri, Ahmed Nabil Belbachir, Tomasz Michalak, Asma Belhadi, and Gautam Srivastava. 2024. Enhancing smart road safety with federated learning for near crash detection to advance the development of the internet of vehicles. *Engineering Applications of Artificial Intelligence* 133 (2024), 108350.
[30] Huong Mai Do, Tuan Phong Tran, and Myungsik Yoo. 2023. Deep reinforcement learning-based task offloading and resource allocation for industrial IoT in MEC federation system. *IEEE Access* 11 (2023), 83150–83170.
[31] Julio C. S. Dos Anjos, João L. G. Gross, Kassiano J. Matteussi, Gabriel V. González, Valderi R. Q. Leithardt, and Claudio F. R. Geyer. 2021. An algorithm to minimize energy consumption and elapsed time for iot workloads in a hybrid architecture. *Sensors* 21, 9 (2021), 2914.
[32] Zhuocheng Du, Yuanzhi Ni, Hongfeng Tao, and Mingfeng Yin. 2024. Joint optimization of offloading strategy and resource allocation for multi-user in dynamic vehicular edge computing systems. *Simulation Modelling Practice and Theory* 136 (2024), 103001.
[33] Rudzidatul Akmam Dziyauddin, Dusit Niyato, Nguyen Cong Luong, Ahmad Ariff Aizuddin Mohd Atan, Mohd Azri Mohd Izhar, Marwan Hadri Azmi, and Salwani Mohd Daud. 2021. Computation offloading and content caching and delivery in vehicular edge network: A survey. *Computer Networks* 197 (2021), 108228.
[34] Ibrahim A Elgendy, Wei-Zhe Zhang, Hui He, Brij B. Gupta, and Ahmed A. Abd El-Latif. 2021. Joint computation offloading and task caching for multi-user and multi-task MEC systems: Reinforcement learning-based algorithms. *Wireless Networks* 27, 3 (2021), 2023–2038.
[35] Scott Ettinger, Shuyang Cheng, Benjamin Caine, Chenxi Liu, Hang Zhao, Sabeek Pradhan, Yuning Chai, Ben Sapp, Charles R. Qi, Yin Zhou, et al. 2021. Large scale interactive motion forecasting for autonomous driving: The waymo open motion dataset. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 9710–9719.
[36] Michael Fallon. 2018. *Self-driving Cars: The New Way Forward*. Twenty-First Century Books™.

[37] Shiyu Fang, Jiaqi Liu, Mingyu Ding, Yiming Cui, Chen Lv, Peng Hang, and Jian Sun. 2025. Towards interactive and learnable cooperative driving automation: A large language model-driven decision-making framework. *IEEE Transactions on Vehicular Technology* 74, 8 (2025), 11894–11905.

[38] Shan Fang, Lan Yang, Wen-long Shang, Xiangmo Zhao, Fengze Li, and Washington Ochieng. 2025. Cooperative control model using reinforcement learning for connected and automated vehicles and traffic signal light at signalized intersections. *IEEE Internet of Things Journal* 12, 21 (2025), 44037–44050.

[39] Adriana Fernández-Fernández, Estefania Coronado, Alberto Erspamer, Georgios Samaras, Vasileios Theodorou, and Shuaib Siddiqui. 2023. Unlocking the path toward intelligent telecom marketplaces for beyond 5g and 6g networks. *IEEE Communications Magazine* 61, 3 (2023), 28–34.

[40] Honghao Gao, Wanqiu Huang, Tong Liu, Yuyu Yin, and Youhuizi Li. 2022. Ppo2: Location privacy-oriented task offloading to edge computing using reinforcement learning for intelligent autonomous transport systems. *IEEE Transactions on Intelligent Transportation Systems* 24, 7 (2022), 7599–7612.

[41] Honghao Gao, Xuejie Wang, Wei Wei, Anwer Al-Dulaimi, and Yueshen Xu. 2023. Com-DDPG: Task offloading based on multiagent reinforcement learning for information-communication-enhanced mobile edge computing in the internet of vehicles. *IEEE Transactions on Vehicular Technology* 73, 1 (2023), 348–361.

[42] Dennis Grewe, Marco Wagner, Mayutan Arumaithurai, Ioannis Psaras, and Dirk Kutscher. 2017. Information-centric mobile edge computing for connected vehicle environments: Challenges and research directions. In *Proceedings of the Workshop on Mobile Edge Communications*. 7–12.

[43] João Luiz Grave Gross, Kassiano José Matteussi, Julio C. S. dos Anjos, and Cláudio Fernando Resin Geyer. 2020. A dynamic cost model to minimize energy consumption and processing time for iot tasks in a mobile edge computing environment. In *Service-Oriented Computing: 18th International Conference, ICSOC 2020, Dubai, United Arab Emirates, December 14–17, 2020, Proceedings 18*. Springer, 101–109.

[44] Ao Guo, Jun Huang, Chen Lv, Long Chen, and Fei-Yue Wang. 2024. Advances in autonomous vehicle testing: The state of the art and future outlook on driving datasets, simulators, and proving grounds. *Authorea Preprints* (2024), 1–64.

[45] Saqib Hakak, Thippa Reddy Gadekallu, Praveen Kumar Reddy Maddikunta, Swarna Priya Ramu, M. Parimala, Chamitha De Alwis, and Madhusanka Liyanage. 2023. Autonomous vehicles in 5G and beyond: A survey. *Vehicular Communications* 39 (2023), 100551.

[46] Yiwen Han, Ding Li, Haotian Qi, Jianji Ren, and Xiaofei Wang. 2019. Federated learning-based computation offloading optimization in edge computing-supported internet of things. In *Proceedings of the ACM Turing Celebration Conference-China*. 1–5.

[47] Haochen Hua, Yutong Li, Tonghe Wang, Nanqing Dong, Wei Li, and Junwei Cao. 2023. Edge computing with artificial intelligence: A machine learning perspective. *Computing Surveys* 55, 9 (2023), 1–35.

[48] Xinyu Huang, Lijun He, and Wanyue Zhang. 2020. Vehicle speed aware computing task offloading and resource allocation based on multi-agent reinforcement learning in a vehicular edge computing network. In *Proceedings of the 2020 IEEE International Conference on Edge Computing*. IEEE, 1–8.

[49] Hatem Ibn-Khedher, Mohammed Laroui, Hassine Moungla, Hossam Afifi, and Emad Abd-Elrahman. 2022. Next-generation edge computing assisted autonomous driving based artificial intelligence algorithms. *IEEE Access* 10 (2022), 53987–54001.

[50] Ali Irshayyid, Jun Chen, and Guojiang Xiong. 2024. A review on reinforcement learning-based highway autonomous vehicle control. *Green Energy and Intelligent Transportation* 3, 4 (2024), 100156.

[51] Zhongming Ji, Li Chen, Nan Zhao, Yunfei Chen, Guo Wei, and F. Richard Yu. 2021. Computation offloading for edge-assisted federated learning. *IEEE Transactions on Vehicular Technology* 70, 9 (2021), 9330–9344.

[52] Yanna Jiang, Baihe Ma, Xu Wang, Guangsheng Yu, Ping Yu, Zhe Wang, Wei Ni, and Ren Ping Liu. 2024. Blockchained federated learning for internet of things: A comprehensive survey. *Computing Surveys* 56, 10 (2024), 1–37.

[53] Roger Kalliomäki. 2019. Real-time object detection for autonomous vehicles using deep learning.

[54] Tadahiko Kawai. 2020. Evaluation of toyota's strategy for connected cars–based on the theory of dynamic managerial capabilities–. *Journal of Strategic Management Studies* 12, 1 (2020), 41–57.

[55] Manzoor Ahmed Khan, Hesham El Sayed, Sumbal Malik, Talha Zia, Jalal Khan, Najla Alkaabi, and Henry Ignatious. 2022. Level-5 autonomous driving–are we there yet? a review of research literature. *ACM Computing Surveys* 55, 2 (2022), 1–38.

[56] Bhoopendra Kumar, Aditya Bhardwaj, and Dinesh Sahu. 2024. Machine learning techniques for task offloading in connected autonomous vehicles. In *Proceedings of the 2024 IEEE 3rd International Conference on Power Electronics, Intelligent Control and Energy Systems*. IEEE, 400–405.

[57] Girish Kumar, Ajith Tom James, Krishna Choudhary, Rishi Sahai, and Weon Keun Song. 2022. Investigation and analysis of implementation challenges for autonomous vehicles in developing countries using hybrid structural modeling. *Technological Forecasting and Social Change* 185 (2022), 122080.

[58] Bo Li, Feilong Chen, Ziyi Peng, Peng Hou, and Hongwei Ding. 2021. Mobility-aware dynamic offloading strategy for C-V2X under multi-access edge computing. *Physical Communication* 49 (2021), 101446.
[59] Chunlin Li, Yong Zhang, and Youlong Luo. 2022. A federated learning-based edge caching approach for mobile edge computing-enabled intelligent connected vehicles. *IEEE Transactions on Intelligent Transportation Systems* 24, 3 (2022), 3360–3369.
[60] Mushu Li, Jie Gao, Lian Zhao, and Xuemin Shen. 2020. Deep reinforcement learning for collaborative edge computing in vehicular networks. *IEEE Transactions on Cognitive Communications and Networking* 6, 4 (2020), 1122–1135.
[61] Meng Li, Bingtong Wang, Shunchao Wang, and Zemian Ke. 2025. Serial distributed reinforcement learning for enhanced multi-objective platoon control in curved road coordinates. *Expert Systems with Applications* 269 (2025), 126493.
[62] Tian Li, Anit Kumar Sahu, Ameet Talwalkar, and Virginia Smith. 2020. Federated learning: Challenges, methods, and future directions. *IEEE Signal Processing Magazine* 37, 3 (2020), 50–60.
[63] Siyuan Liang, Hao Wu, Li Zhen, Qiaozhi Hua, Sahil Garg, Georges Kaddoum, Mohammad Mehedi Hassan, and Keping Yu. 2022. Edge YOLO: Real-time intelligent object detection system based on edge-cloud cooperation in autonomous vehicles. *IEEE Transactions on Intelligent Transportation Systems* 23, 12 (2022), 25345–25360.
[64] Bing Lin, Kai Lin, Changhang Lin, Yu Lu, Ziqing Huang, and Xinwei Chen. 2021. Computation offloading strategy based on deep reinforcement learning for connected and autonomous vehicle in vehicular edge computing. *Journal of Cloud Computing* 10, 1 (2021), 33.
[65] Hai Lin, Sherali Zeadally, Zhihong Chen, Houda Labiod, and Lusheng Wang. 2020. A survey on computation offloading modeling for edge computing. *Journal of Network and Computer Applications* 169 (2020), 102781.
[66] Joe Lindsey. 2024. Self-driving cars vs. the world. *Popular Mechanics* (2024), 44–58.
[67] Guozhi Liu, Fei Dai, Bi Huang, Zhenping Qiang, Shuai Wang, and Lecheng Li. 2022. A collaborative computation and dependency-aware task offloading method for vehicular edge computing: A reinforcement learning approach. *Journal of Cloud Computing* 11, 1 (2022), 68.
[68] Pengju Liu, Junluo Li, and Zhongwei Sun. 2019. Matching-based task offloading for vehicular edge computing. *IEEE Access* 7 (2019), 27628–27640.
[69] Yongkang Liu, Chianing Wang, and Kentaro Oguchi. 2023. Poster: Towards realistic federated learning evaluations for connected and automated vehicles. In *Proceedings of the 2023 IEEE/ACM Symposium on Edge Computing*. IEEE, 244–246.
[70] Ziye Liu, Chen Chen, Zhiyi Wang, Li Cong, Haitao Lu, Qingqi Pei, and Shaohua Wan. 2024. Automated vehicle platooning: A two-stage approach based on vehicle-road cooperation. *IEEE Transactions on Intelligent Vehicles* (2024), 1–15.
[71] Minghui Liwang, Jiexiang Wang, Zhibin Gao, Xiaojiang Du, and Mohsen Guizani. 2019. Game theory based opportunistic computation offloading in cloud-enabled IoV. *IEEE Access* 7 (2019), 32551–32561.
[72] Quyuan Luo, Tom H Luan, Weisong Shi, and Pingzhi Fan. 2022. Deep reinforcement learning based computation offloading and trajectory planning for multi-UAV cooperative target search. *IEEE Journal on Selected Areas in Communications* 41, 2 (2022), 504–520.
[73] Pin Lv, Wenbiao Xu, Jiangtian Nie, Yanli Yuan, Chao Cai, Zhe Chen, and Jia Xu. 2022. Edge computing task offloading for environmental perception of autonomous vehicles in 6g networks. *IEEE Transactions on Network Science and Engineering* 10, 3 (2022), 1228–1245.
[74] Stefano Mariani, Giacomo Cabri, and Franco Zambonelli. 2021. Coordination of autonomous vehicles: Taxonomy and survey. *ACM Computing Surveys* 54, 1 (2021), 1–33.
[75] Adila Mebrek and Abdulsalam Yassine. 2021. Intelligent resource allocation and task offloading model for IoT applications in fog networks: A game-theoretic approach. *IEEE Transactions on Emerging Topics in Computational Intelligence* (2021), 1–15.
[76] Leo Mendiboure, Mohamed-Aymen Chalouf, and Francine Krief. 2019. Edge computing based applications in vehicular environments: Comparative study and main issues. *Journal of Computer Science and Technology* 34, 4 (2019), 869–886.
[77] Rodolfo Meneguette, Robson De Grande, Jo Ueyama, Geraldo P. Rocha Filho, and Edmundo Madeira. 2021. Vehicular edge computing: Architecture, resource management, security, and challenges. *ACM Computing Surveys* 55, 1 (2021), 1–46.
[78] Haitao Min, Xiaoyong Xiong, Fang Yang, Weiyi Sun, Yuanbin Yu, and Pengyu Wang. 2023. An energy-efficient driving method for connected and automated vehicles based on reinforcement learning. *Machines* 11, 2 (2023), 168.
[79] Komeil Moghaddasi and Shakiba Rajabi. 2024. Blockchain-enhanced offloading in mobile edge computing: A systematic review and survey of current trends and future directions. 1–37. arXiv:2403.05961. Retrieved from https://arxiv.org/abs/2403.05961
[80] Mohammad S. Mohammed, Ali M. Abduljabar, Mustafa M. Faisal, Basheera M. Mahmmod, Sadiq H. Abdulhussain, Wasiq Khan, Panos Liatsis, and Abir Hussain. 2023. Low-cost autonomous car level 2: Design and implementation for conventional vehicles. *Results in Engineering* 17 (2023), 100969.

[81] Sungwon Moon and Yujin Lim. 2022. Federated deep reinforcement learning based task offloading with power control in vehicular edge computing. *Sensors* 22, 24 (2022), 9595.
[82] Naren, Abhishek Kumar Gaurav, Nishad Sahu, Abhinash Prasad Dash, GSS Chalapathi, and Vinay Chamola. 2022. A survey on computation resource allocation in IoT enabled vehicular edge computing. *Complex and Intelligent Systems* 8, 5 (2022), 1–23.
[83] Khoa Nguyen, Steve Drew, Changcheng Huang, and Jiayu Zhou. 2020. Collaborative container-based parked vehicle edge computing framework for online task offloading. In *Proceedings of the 2020 IEEE 9th International Conference on Cloud Networking*. IEEE, 1–6.
[84] Khoa Nguyen, Steve Drew, Changcheng Huang, and Jiayu Zhou. 2022. Parked vehicles task offloading in edge computing. *IEEE Access* 10 (2022), 41592–41606.
[85] NMSC. [n. d.]. Autonomous Vehicle Market. Retrieved September 20, 2024 from https://www.nextmsc.com/report/autonomous-vehicle-market
[86] Level of automation. 2024. *National Highway Traffic Safety Administration (NHTSA)*. Retrieved April 20, 2024 from https://www.nhtsa.gov/
[87] Pratham Oza, Nathaniel Hudson, Thidapat Chantem, and Hana Khamfroush. 2024. Deadline-aware task offloading for vehicular edge computing networks using traffic light data. *ACM Transactions on Embedded Computing Systems* 23, 1 (2024), 1–25.
[88] Budi Padmaja, C. H. VKNSN Moorthy, N. Venkateswarulu, and Myneni Madhu Bala. 2023. Exploration of issues, challenges and latest developments in autonomous cars. *Journal of Big Data* 10, 1 (2023), 61.
[89] Srikanta Pradhan, Somanath Tripathy, and Rakesh Matam. 2024. Towards optimal edge resource utilization: Predictive analytics and reinforcement learning for task offloading. *Internet of Things* 26 (2024), 101147.
[90] Sahaya Beni Prathiba, Gunasekaran Raja, Sudha Anbalagan, Kapal Dev, Sugeerthi Gurumoorthy, and Atshaya P. Sankaran. 2021. Federated learning empowered computation offloading and resource management in 6G-V2X. *IEEE Transactions on Network Science and Engineering* 9, 5 (2021), 3234–3243.
[91] Qi Qi, Lingxin Zhang, Jingyu Wang, Haifeng Sun, Zirui Zhuang, Jianxin Liao, and F. Richard Yu. 2020. Scalable parallel task scheduling for autonomous driving using multi-task deep reinforcement learning. *IEEE Transactions on Vehicular Technology* 69, 11 (2020), 13861–13874.
[92] Mostafa Raeisi-Varzaneh, Omar Dakkak, Adib Habbal, and Byung-Seo Kim. 2023. Resource scheduling in edge computing: Architecture, taxonomy, open issues and future research directions. *IEEE Access* 11 (2023), 25329–25350.
[93] Salman Raza, Shangguang Wang, Manzoor Ahmed, and Muhammad Rizwan Anwar. 2019. A survey on vehicular edge computing: Architecture, applications, technical issues, and future directions. *Wireless Communications and Mobile Computing* 2019, 1 (2019), 3159762.
[94] Insaf Rzig, Wael Jafaar, Maha Jebalia, and Sami Tabbane. 2024. Dependency-aware task offloading in cooperative UAV-HAPS-assisted vehicular networks. In *Proceedings of the 2024 International Wireless Communications and Mobile Computing*. IEEE, 1541–1546.
[95] Firdose Saeik, Marios Avgeris, Dimitrios Spatharakis, Nina Santi, Dimitrios Dechouniotis, John Violos, Aris Leivadeas, Nikolaos Athanasopoulos, Nathalie Mitton, and Symeon Papavassiliou. 2021. Task offloading in edge and cloud computing: A survey on mathematical, artificial intelligence and control theory solutions. *Computer Networks* 195 (2021), 108177.
[96] Nischal Sanil, V. Rakesh, Rishab Mallapur, and Mohammed Riyaz Ahmed. 2020. Deep learning techniques for obstacle detection and avoidance in driverless cars. In *Proceedings of the 2020 International Conference on Artificial Intelligence and Signal Processing*. IEEE, 1–4.
[97] F. B. Scurt, T. Vesselenyi, R. C. Tarca, H. Beles, and G. Dragomir. 2021. Autonomous vehicles: Classification, technology and evolution. In *Proceedings of the IOP Conference Series: Materials Science and Engineering*. IOP Publishing, 012032.
[98] Vivek Sethi and Sujata Pal. 2023. FedDOVe: A federated deep Q-learning-based offloading for vehicular fog computing. *Future Generation Computer Systems* 141 (2023), 96–105.
[99] Caixing Shao, Fengxin Cheng, Jingzhong Xiao, and Ke Zhang. 2023. Vehicular intelligent collaborative intersection driving decision algorithm in Internet of Vehicles. *Future Generation Computer Systems* 145 (2023), 384–395.
[100] Surbhi Sharma and Baijnath Kaushik. 2019. A survey on internet of vehicles: Applications, security issues and solutions. *Vehicular Communications* 20 (2019), 100182.
[101] Swapnil Sadashiv Shinde, Arash Bozorgchenani, Daniele Tarchi, and Qiang Ni. 2021. On the design of federated learning in latency and energy constrained computation offloading operations in vehicular edge computing systems. *IEEE Transactions on Vehicular Technology* 71, 2 (2021), 2041–2057.
[102] Nirajan Shiwakoti, Peter Stasinopoulos, and Francesco Fedele. 2020. Investigating the state of connected and autonomous vehicles: A literature Review. *Transportation Research Procedia* 48 (2020), 870–882.
[103] Aggelos Soteropoulos, Mathias Mitteregger, Martin Berger, and Jakob Zwirchmayr. 2020. Automated drivability: Toward an assessment of the spatial deployment of level 4 automated vehicles. *Transportation Research Part A: Policy and Practice* 136 (2020), 64–84.

[104] Thilo Strauss, Michael Oechsle, and Uwe Bauknecht. 2024. Differentiable optimization for orchestration: Resource offloading for vehicles in smart cities. *IEEE Access* 12 (2021), 23798–23807.
[105] Jian Su, Jinguo Pan, Xiukai Ruan, and Xuedong Zhang. 2023. RVEAPE: An approach to computation offloading for connected autonomous vehicles. *IEEE Transactions on Automation Science and Engineering* 21, 3 (2023), 2412–2424.
[106] Ebrahim Hamid Sumiea, Said Jadid Abdulkadir, Hitham Seddig Alhussian, Safwan Mahmood Al-Selwi, Alawi Alqushaibi, Mohammed Gamal Ragab, and Suliman Mohamed Fati. 2024. Deep deterministic policy gradient algorithm: A systematic review. *Heliyon* 10, 9 (2024), 1–26.
[107] Jianan Sun, Qing Gu, Tao Zheng, Ping Dong, Alvin Valera, and Yajuan Qin. 2020. Joint optimization of computation offloading and task scheduling in vehicular edge computing networks. *IEEE Access* 8 (2020), 10466–10477.
[108] Shujuan Tian, Xianghong Deng, Pengpeng Chen, Tingrui Pei, Sangyoon Oh, and Weiping Xue. 2021. A dynamic task offloading algorithm based on greedy matching in vehicle network. *Ad Hoc Networks* 123 (2021), 102639.
[109] Ashab Uddin, Ahmed Hamdi Sakr, and Ning Zhang. 2024. Adaptive prioritization and task offloading in vehicular edge computing through deep reinforcement learning. *IEEE Transactions on Vehicular Technology* 74, 3 (2024), 5038–5052.
[110] Blesson Varghese and Rajkumar Buyya. 2018. Next generation cloud computing: New trends and research directions. *Future Generation Computer Systems* 79 (2018), 849–861.
[111] Hansong Wang, Xi Li, Hong Ji, and Heli Zhang. 2018. Federated offloading scheme to minimize latency in MEC-enabled vehicular networks. In *Proceedings of the 2018 IEEE Globecom Workshops*. IEEE, 1–6.
[112] Jin Wang, Jia Hu, Geyong Min, Albert Y. Zomaya, and Nektarios Georgalas. 2020. Fast adaptive task offloading in edge computing based on meta reinforcement learning. *IEEE Transactions on Parallel and Distributed Systems* 32, 1 (2020), 242–253.
[113] Lei Wang, Hongbin Liang, and Dongmei Zhao. 2023. Deep reinforcement learning-based computation offloading and power allocation within dynamic platoon network. *IEEE Internet of Things Journal* 11, 6 (2023), 10500–10512.
[114] Zekun Wang, Jin Nakazato, Muhammad Asad, Ehsan Javanmardi, and Manabu Tsukada. 2023. Overcoming environmental challenges in CAVs through MEC-based federated learning. In *Proceedings of the 2023 14th International Conference on Ubiquitous and Future Networks*. IEEE, 151–156.
[115] Zhen Wang, Sifa Zheng, Qiang Ge, and Keqiang Li. 2020. Online offloading scheduling and resource allocation algorithms for vehicular edge computing system. *IEEE Access* 8 (2020), 52428–52442.
[116] Seokkyun Woo, Jan Youtie, Ingrid Ott, and Fenja Scheu. 2021. Understanding the long-term emergence of autonomous vehicles technologies. *Technological Forecasting and Social Change* 170 (2021), 120852.
[117] Sijun Wu, Liang Yang, Junjie Li, Hongzhi Guo, Ishtiaq Ahmad, Daniel Benevides Da Costa, Hongbo Jiang, and Dusit Niyato. 2025. DRL-based pricing-driven for task offloading and dynamic resource in vehicle edge computing. *IEEE Transactions on Mobile Computing* 24, 10 (2025), 10389–10404.
[118] Zhu Xiao, Jinmei Shu, Hongbo Jiang, Geyong Min, Hongyang Chen, and Zhu Han. 2022. Perception task offloading with collaborative computation for autonomous driving. *IEEE Journal on Selected Areas in Communications* 41, 2 (2022), 457–473.
[119] Kai Xiong, Supeng Leng, Chongwen Huang, Chau Yuen, and Yong Liang Guan. 2020. Intelligent task offloading for heterogeneous V2X communications. *IEEE Transactions on Intelligent Transportation Systems* 22, 4 (2020), 2226–2238.
[120] Guang Yang, Pengqi Wang, Yuan Shi, Qian Dong, and Weiyu Liu. 2022. A high precision annotation method for vision simulation data of autonomous vehicle. In *Proceedings of the 2022 9th International Conference on Dependable Systems and Their Applications*. IEEE, 738–745.
[121] Jingxuan Yang, Ruoxuan Bai, Haoyuan Ji, Yi Zhang, Jianming Hu, and Shuo Feng. 2025. Adaptive testing environment generation for connected and automated vehicles with dense reinforcement learning. *IEEE Transactions on Intelligent Transportation Systems* 26, 4 (2025), 5135–5145.
[122] Yang Yang, Caixing Shao, Jinhui Zuo, and Chunwen Shi. 2024. Energy efficient algorithm for multi-user adaptive edge computing offloading in vehicular networks based on meta reinforcement learning. In *Proceedings of the 2024 7th World Conference on Computing and Communication Technologies*. IEEE, 250–254.
[123] Takahito Yoshizawa, Dave Singelée, Jan Tobias Muehlberg, Stéphane Delbruel, Amir Taherkordi, Danny Hughes, and Bart Preneel. 2023. A survey of security and privacy issues in v2x communication systems. *Computing Surveys* 55, 9 (2023), 1–36.
[124] Maram Bani Younes and Azzedine Boukerche. 2020. Traffic efficiency applications over downtown roads: A new challenge for intelligent connected vehicles. *ACM Computing Surveys* 53, 5 (2020), 1–30.
[125] Zeinab Zabihi, Amir Masoud Eftekhari Moghadam, and Mohammad Hossein Rezvani. 2023. Reinforcement learning methods for computation offloading: A systematic review. *Computing Surveys* 56, 1 (2023), 1–41.
[126] Feng Zeng, Zheng Zhang, and Jinsong Wu. 2024. Task offloading delay minimization in vehicular edge computing based on vehicle trajectory prediction. *Digital Communications and Networks* 11, 2 (2024), 537–546.
[127] Qixun Zhang, Hao Wen, Ying Liu, Shuo Chang, and Zhu Han. 2022. Federated-reinforcement-learning-enabled joint communication, sensing, and computing resources allocation in connected automated vehicles networks. *IEEE Internet of Things Journal* 9, 22 (2022), 23224–23240.

[128] Siqi Zhang, Na Yi, and Yi Ma. 2024. A survey of computation offloading with task types. *IEEE Transactions on Intelligent Transportation Systems* 25, 8 (2024), 8313–8333.
[129] Wenhan Zhang, Mingjie Feng, and Marwan Krunz. 2023. Latency estimation and computational task offloading in vehicular mobile edge computing applications. *IEEE Transactions on Vehicular Technology* 73, 4 (2023), 5808–5823.
[130] Yuru Zhang, Qiang Liu, Haoxin Wang, Dawei Chen, and Kyungtae Han. 2024. Crowdsourcing live high definition map via collaborative computation in automotive edge computing. *IEEE Transactions on Vehicular Technology* 73, 9 (2024), 13569–13583.
[131] Chen Ziyan and Liu Shiguo. 2021. China's self-driving car legislation study. *Computer Law and Security Review* 41 (2021), 105555.

Received 1 October 2024; revised 27 November 2025; accepted 3 December 2025